

RUHR-UNIVERSITÄT BOCHUM

## **Evaluierung der Sicherheit von JavaScript in PDFs an dem Beispiel von Adobe Acrobat Reader DC**

Christian Pressler

Bachelorarbeit – 17. Januar 2020.  
Lehrstuhl für Netz- und Datensicherheit.

Supervisor: Prof. Dr. Jörg Schwenk  
Advisor: Dr.-Ing. Vladislav Mladenov  
Dr.-Ing. Christian Mainka



## **Zusammenfassung**

PDF ist ein weltweit verwendetes Dateiformat und meistens die erste Wahl, wenn es um den Austausch von elektronischen Dokumenten geht. Die Integration von JavaScript durch die von Adobe Produkten gestellte JavaScript API bietet unter anderem die Möglichkeit, Eingabevalidierungen in digitalen Formularen vorzunehmen, Verbindungen mit Datenbanken zu erstellen, Sicherheitsrichtlinien zu verwalten und das Aussehen, wie auch den Inhalt einer PDF-Datei dynamisch zu verändern. Diese und weitere JavaScript basierten Aktionen werden durch entsprechende API-Aufrufe ausgeführt, die in dieser Arbeit auf ihre Funktionalität und Anwendbarkeit in Hinsicht auf die Sicherheit untersucht und getestet werden.



## Official Declaration

Hereby I declare, that I have not submitted this thesis in this or similar form to any other examination at the Ruhr-Universität Bochum or any other institution or university.

I officially ensure, that this paper has been written solely on my own. I hereby officially ensure, that I have not used any other sources but those stated by me. Any and every parts of the text which constitute quotes in original wording or in its essence have been explicitly referred by me by using official marking and proper quotation. This is also valid for used drafts, pictures and similar formats.

I also officially ensure that the printed version as submitted by me fully confirms with my digital version. I agree that the digital version will be used to subject the paper to plagiarism examination.

Not this English translation, but only the official version in German is legally binding.

## Eidesstattliche Erklärung

Ich erkläre, dass ich keine Arbeit in gleicher oder ähnlicher Fassung bereits für eine andere Prüfung an der Ruhr-Universität Bochum oder einer anderen Hochschule eingereicht habe.

Ich versichere, dass ich diese Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen benutzt habe. Die Stellen, die anderen Quellen dem Wortlaut oder dem Sinn nach entnommen sind, habe ich unter Angabe der Quellen kenntlich gemacht. Dies gilt sinngemäß auch für verwendete Zeichnungen, Skizzen, bildliche Darstellungen und dergleichen.

Ich versichere auch, dass die von mir eingereichte schriftliche Version mit der digitalen Version übereinstimmt. Ich erkläre mich damit einverstanden, dass die digitale Version dieser Arbeit zwecks Plagiatsprüfung verwendet wird.

---

DATE

---

CHRISTIAN PRESSLER



## Erklärung

Ich erkläre mich damit einverstanden, dass meine Bachelorarbeit am Lehrstuhl NDS dauerhaft in elektronischer und gedruckter Form aufbewahrt wird und dass die Ergebnisse aus dieser Arbeit unter Einhaltung guter wissenschaftlicher Praxis in der Forschung weiter verwendet werden dürfen.

---

DATE

---

CHRISTIAN PRESSLER



# Inhaltsverzeichnis

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                                                  | <b>1</b>  |
| 1.1      | Motivation . . . . .                                               | 2         |
| 1.2      | Stand der Forschung . . . . .                                      | 3         |
| 1.3      | Struktur der Bachelorarbeit . . . . .                              | 5         |
| <b>2</b> | <b>Grundlagen</b>                                                  | <b>7</b>  |
| 2.1      | Begriffserklärungen . . . . .                                      | 7         |
| 2.1.1    | SDK - Software Development Kit . . . . .                           | 7         |
| 2.1.2    | API - Application Programming Interface . . . . .                  | 7         |
| 2.1.3    | API-Aufruf . . . . .                                               | 8         |
| 2.2      | PDFs im Allgemeinen . . . . .                                      | 8         |
| 2.3      | JavaScript in PDFs . . . . .                                       | 9         |
| 2.4      | JavaScript Events . . . . .                                        | 10        |
| <b>3</b> | <b>Sicherheitskritische API-Aufrufe</b>                            | <b>13</b> |
| 3.1      | JavaScript für Acrobat API Referenz . . . . .                      | 13        |
| 3.2      | Wichtige Objekte . . . . .                                         | 13        |
| 3.3      | Angriffsszenario . . . . .                                         | 15        |
| 3.4      | Auswahlkriterien . . . . .                                         | 15        |
| 3.5      | Die Kategorien . . . . .                                           | 16        |
| 3.5.1    | Kategorie 1: Externe Zugriffe . . . . .                            | 16        |
| 3.5.1.1  | URI-Aufrufe . . . . .                                              | 16        |
| 3.5.1.2  | Netzwerkdienste . . . . .                                          | 17        |
| 3.5.1.3  | Dateisystemzugriffe . . . . .                                      | 18        |
| 3.5.1.4  | Programmausführung . . . . .                                       | 19        |
| 3.5.1.5  | Ausführung einer beliebigen Datei mit Dateisystemzugriff . . . . . | 19        |
| 3.5.2    | Kategorie 2: Anwendungsinteraktionen . . . . .                     | 20        |
| 3.5.2.1  | UI-Manipulation . . . . .                                          | 20        |
| 3.5.2.2  | Anwendungseigenschaftenmanipulation . . . . .                      | 21        |
| 3.5.2.3  | UI-Interaktion . . . . .                                           | 22        |
| 3.5.2.4  | Ausführung einer Anwendungsaktion . . . . .                        | 23        |
| 3.5.2.5  | Automatisierte Codeausführung . . . . .                            | 23        |
| 3.5.3    | Kategorie 3: Offenlegung von Informationen . . . . .               | 23        |
| 3.5.4    | Kategorie 4: Dokumentmanipulation . . . . .                        | 24        |
| 3.5.4.1  | Dokumenteigenschaftenmanipulation . . . . .                        | 25        |

|          |                                                                                          |           |
|----------|------------------------------------------------------------------------------------------|-----------|
| 3.5.4.2  | Dokumentinhaltmanipulation . . . . .                                                     | 26        |
| 3.5.4.3  | Dokumentinhaltmanipulation mit Dateisystemzugriff                                        | 27        |
| 3.5.4.4  | Feldmanipulation . . . . .                                                               | 27        |
| 3.5.4.5  | Annotationmanipulation . . . . .                                                         | 29        |
| 3.5.4.6  | Annotationmanipulation mit Dateisystemzugriff . .                                        | 30        |
| 3.5.4.7  | Verzögerte Ausführungen . . . . .                                                        | 30        |
| 3.5.4.8  | Bookmarks . . . . .                                                                      | 31        |
| 3.5.4.9  | Zugriff auf andere Dokumente . . . . .                                                   | 31        |
| 3.5.4.10 | Temporäres Abspeichern eines Dokumentinhalts . .                                         | 31        |
| 3.5.5    | Kategorie 5: Rechteauserweiterung . . . . .                                              | 32        |
| <b>4</b> | <b>Testaufbau</b>                                                                        | <b>33</b> |
| 4.1      | Sicherheitsmechanismen . . . . .                                                         | 33        |
| 4.1.1    | Privilegierter Kontext . . . . .                                                         | 33        |
| 4.1.2    | Secure Path . . . . .                                                                    | 34        |
| 4.2      | Testumgebung . . . . .                                                                   | 34        |
| 4.2.1    | Software . . . . .                                                                       | 34        |
| 4.2.2    | Standardeinstellungen für JavaScript im Adobe Acrobat Reader DC . . . . .                | 34        |
| 4.3      | Durchführung . . . . .                                                                   | 35        |
| 4.3.1    | Erstellung der PDF-Dateien . . . . .                                                     | 35        |
| 4.3.2    | Erstellung der Zertifikate . . . . .                                                     | 35        |
| 4.3.3    | Zertifikate hinzufügen . . . . .                                                         | 36        |
| 4.3.4    | Testfälle . . . . .                                                                      | 37        |
| 4.3.5    | Testablauf . . . . .                                                                     | 37        |
| <b>5</b> | <b>Testergebnisse</b>                                                                    | <b>39</b> |
| 5.1      | Sicherheitswarnungen . . . . .                                                           | 39        |
| 5.2      | Auswertung . . . . .                                                                     | 39        |
| 5.2.1    | Testfall 1: PDF-Datei ohne Zertifikat . . . . .                                          | 40        |
| 5.2.2    | Testfall 2: Zertifikat erlaubt dynamische Inhalte . . . . .                              | 43        |
| 5.2.3    | Testfall 3: Zertifikat erlaubt JavaScript-Skripte mit hoher Berechtigungsstufe . . . . . | 43        |
| 5.2.4    | Testfall 4: Zertifikat erlaubt privilegierte Systemvorgänge . .                          | 45        |
| 5.2.5    | Gemeinsamkeiten von Testfall 3 und 4 . . . . .                                           | 46        |
| 5.2.6    | Nicht ausführbare API-Aufrufe . . . . .                                                  | 47        |
| <b>6</b> | <b>Zusammenfassung</b>                                                                   | <b>49</b> |
| <b>7</b> | <b>Fazit und Ausblick</b>                                                                | <b>53</b> |
|          | <b>Abbildungsverzeichnis</b>                                                             | <b>55</b> |
|          | <b>Tabellenverzeichnis</b>                                                               | <b>56</b> |

|                                 |           |
|---------------------------------|-----------|
| <i>Inhaltsverzeichnis</i>       | xi        |
| <b>Liste der Listings</b>       | <b>57</b> |
| <b>Literaturverzeichnis</b>     | <b>58</b> |
| <b>A Anhang: Testergebnisse</b> | <b>61</b> |



# 1 Einleitung

Das Portable Document Format (PDF) ist ein beliebtes Dateiformat, das von Adobe für die Erstellung und den Austausch von Dokumenten entwickelt wurde. Das Dateiformat weist universelle Einsetzbarkeit auf, da es unabhängig von Anwendungssoftware, Geräten und Betriebssystemen ist. Dadurch ist es möglich, Dokumente genauso darzustellen, wie sie ursprünglich erstellt wurden, da sie ein einheitliches Layout mit Text, Schriften, Vektorgrafiken, Rasterbildern und anderen anzuzeigenden Informationen speichern können. Die genannten Eigenschaften führten zum großen Erfolg des PDF und zu dessen weltweiten Verbreitung. Seit 2008 ist es als offenes Format (ISO 32000) standardisiert.

Im Laufe der Zeit wurden neue Funktionen hinzugefügt, wie die Integration von JavaScript. Dies ist eine Skriptsprache, die ursprünglich für dynamische Inhalte in Webbrowsern entwickelt wurde. Die Skripte werden clientseitig, das bedeutet auf dem System des Benutzers, ausgeführt. Dadurch kann der Inhalt und die Darstellung einer Webseite verändert werden. Zum selben Zweck wird JavaScript in PDF-Dateien eingesetzt.

Die folgende Bachelorarbeit beschäftigt sich mit der Evaluierung der Sicherheitsrisiken bei der Verwendung von JavaScript in PDF-Dateien im Adobe Acrobat Reader DC. JavaScript ist eine vielseitige und flexible Sprache, die unter Anderem bereits bekannt für einige Angriffe ist, beispielsweise Cross-Site-Skripting (XSS). Deshalb wird davon ausgegangen, dass auch in diesem Fall sicherheitskritische Aktionen ausgeführt werden können. Dabei werden unter Anderem Funktionen gesucht, die Dokumentinhalte nachträglich ändern können und somit die Integrität eines Dokuments beeinträchtigen. Ein weiterer Aspekt ist die Ausführung sicherheitskritischer Aktionen, wie Zugriffe auf das Dateisystem oder Manipulationen der Applikation.

Die Evaluierung ist in zwei Teile gegliedert und beginnt mit der Untersuchung einsetzbarer JavaScript-Methoden. Sie sind in der Acrobat DC SDK Dokumentation spezifiziert und werden in Hinsicht auf die Sicherheit unter der Verwendung des Adobe Acrobat Reader DC untersucht. Die sicherheitskritischen API-Aufrufe werden im Anschluss in Kategorien aufgeteilt.

Der zweite Teil beschäftigt sich zunächst mit dem Entwurf eines systematischen Testablaufs. Anschließend werden die API-Aufrufe, die während dieser Arbeit als sicherheitskritisch eingestuft wurden, getestet und bewertet.

## 1.1 Motivation

PDF-Dateien sind populär und werden zum Austausch von Daten benutzt, sei es im gewerblichen oder im privaten Umfeld. Im Oktober 2015 schätzte Adobe Systems Vice President of Engineering, Phil Ydens, auf der PDF Techincal Conference, dass zu dem Zeitpunkt ungefähr 1.6 Milliarden PDF-Dokumente im Internet verfügbar waren [22]. Weiterhin schätzt er die Anzahl, der jährlich erstellten PDF-Dokumente auf 2.5 Billionen. Dadurch ist es ein attraktives Ziel für Angreifer, zu versuchen, Dokumente zu fälschen oder nach Sicherheitslücken in der Reader Applikation zu suchen.

Seit der Veröffentlichung des Dateiformats 1993 wurde die Spezifikation weiterentwickelt und um neue Funktionen erweitert. Dazu zählt beispielsweise die Möglichkeit, Formularfelder zu erstellen oder 3D-Objekte einzubinden. Eine nützliche Neuerung stellt die Integration von JavaScript dar. Diese Integration bietet den Anwendern eine Reihe von Möglichkeiten, PDF Dateien zu gestalten. Dadurch ist es nun unter anderem möglich, interaktive Dokumente wie auch digitale, ausfüllbare Formulare mit Benutzereingabevalidierung zu erstellen. Auch kann JavaScript dazu genutzt werden, Aktionen beim Öffnen einer Datei ausführen zu lassen, beispielsweise das Öffnen einer URL. So könnte ein Angreifer einen Benutzer direkt auf seine schadhafte Webseite leiten.

Offensichtlich lässt sich JavaScript auch für bösartige Zwecke ausnutzen, wodurch schon 2006 PDF-Dokumente als beliebte Angriffsvektoren gehandelt wurden [2]. Die deutschen Forscher von AV-Test verzeichneten 2013 nach einer 10-jährigen Langzeitstudie etwa 37.000 registrierte Exploitvarianten für den Adobe Reader, der damit den Spitzenreiter unter den Malware-Einfallstoren stellte [18].

Zu der Zeit schien es noch kein probates Mittel gegen diese Angriffe zu geben. Das Problem erkannte unter Anderen auch die Forschungsgruppe um Pavel Laskov und Nedim Srndic [15] und entwickelte darauf hin eine Software, um schadhafte Code in PDF Dateien zu entdecken. Dabei galt es, eine gute Methodologie zu entwickeln, damit das Programm schnell, zuverlässig und benutzerfreundlich bleibt. Es wurde dafür vorher festgelegt, wo sich der schadhafte Code befinden kann und wie er aussieht. Die JavaScript API hat sich seit dem weiterentwickelt und die Ergebnisse dieser Arbeit könnten verwendet werden, um derartige Software zu optimieren.

Für die Erforschung von schadhaftem Code innerhalb von PDF-Dateien mit JavaScript ist es notwendig zu wissen, welche Schwachstellen die Reader mit sich bringen und wie man diese ausnutzen kann. Die aktuellsten Sicherheitslücken<sup>1</sup>, für die es schon ein Update gibt, hat Adobe auf ihrer Webseite veröffentlicht.

Adobe bietet eine eigene API für JavaScript an, die für ihre Produkte genutzt werden kann. Dabei gibt es viele Funktionen, die von Anwendern verwendet werden können, um ihre Dokumente einfacher zu erstellen und beim Empfänger bestimmte Aktionen

---

<sup>1</sup><https://helpx.adobe.com/security/products/acrobat/apsb19-55.html>

ausführen zu lassen. In dieser Arbeit sollen genau diese von Adobe Produkten spezifischen JavaScript Funktionalitäten in Bezug auf Sicherheitsrisiken untersucht werden.

## 1.2 Stand der Forschung

2010 erschien ein Artikel über die Sicherheit und den Datenschutz bezüglich PDF-Dateien unter der Verwendung von verschiedener Software [3]. Bereits damals war schon bekannt, dass die Programme, mit denen man PDF Dateien liest und bearbeitet, gewisse Schwachstellen vorweisen. Das Ausnutzen von JavaScript in PDF Dateien wurde getestet, indem Ereignisse (siehe Abschnitt 2.4), die beim Öffnen einer Datei schadhafte Code ausführen, in PDFs eingebettet wurden. Der schadhafte Code schickte einen Request an eine URI, sodass erkennbar war, wenn eine Datei geöffnet wurde. Anschließend verschickten die Forscher die präparierten PDFs an einige Studenten und Professoren und erhielten von fast jeder versendeten Datei eine Rückmeldung. Danach wurde ein weiterer alternativer Angriff vorgestellt, welcher das PDF Dokument mit einem *Fully Qualified Domain Name* verknüpft und dadurch die DNS-Auflösung austrickst.

Die Angriffstechniken wurden immer komplexer und nutzten verschiedene Schwachstellen in den Readern aus. Dagegen entwickelten einige Forschungsgruppen Tools, um schadhafte PDF Dateien frühzeitig zu erkennen [15, 21, 6, 16]. PJScan wurde 2011 entwickelt und ist ein statischer PDF-Scanner [15]. Dabei sucht die Software nach Schlüsselwörtern wie /JS, /S, /JavaScript oder /Rendition, da an diesen Stellen JavaScript Code eingebettet sein kann. Die Analyse wird lexikalisch ausgeführt und benutzt *maschinelles Lernen*, um Modelle aus verfügbaren Daten für eine spätere Klassifizierung neuer Daten zu erstellen.

Andere Forscher entwickelten ein Framework, das schadhafte Dokumente durch die Metadaten und Struktur erkennt [21]. Ihre Untersuchung ergab, dass schadhafte PDF-Dokumente immer gewisse Gemeinsamkeiten aufweisen, auch wenn sie andere Schwachstellen ausnutzen. Sie unterscheiden sich jedoch deutlich zu normalen Dokumenten. Das Prinzip ist ähnlich zu Spam-Filtern und beruht auf statistischem Lernen.

2014 verfolgten Daiping Liu, Haining Wang und Angelos Stavrou einen anderen Ansatz in ihrer Arbeit, um schadhafte JavaScript-Code in PDF Dateien zu erkennen. Sie verfolgten eine hybride Herangehensweise aus dynamischen und statischen Elementen [16]. Dabei wird als dynamischer Teil Kontext-Monitoring-Code in das Dokument implementiert und statisch nach verdächtigen Eigenschaften gesucht. Auch hier kommt wieder das maschinelle Lernen zum Einsatz, wodurch statische Modelle von Dokumentinhalten entworfen werden.

Nur einige Monate später entwickelten vier Forscher aus Italien ein weiteres Werkzeug, um JS Malware aufzudecken [6]. Jedoch bemängelten sie die vorherigen Herangehensweisen mit maschinellem Lernen, da diese Technik eine hohe Anzahl an falschen Meldungen produzierte. Stattdessen exploiteten sie diese Techniken, um eine Untermenge an API Referenzen zu erstellen, die schadhafte Code kennzeichnet, woraus dann ein Schwachstellendetektor entworfen wurde. Die Forschung basiert dabei auf der API Referenz von Adobe, die der Hauptbestandteil der Untersuchung in dieser Arbeit ist.

Die Forscher Davide Maiorca und Battista Biggio stellten fest, dass schadhafte PDF-Dokumente typischerweise JavaScript-Code, kodierte Datenströme oder eingebettete Objekte, wie Bilder oder ActionScript-Code enthalten, um Schwachstellen im Adobe Reader auszunutzen [17]. In ihrer Arbeit untersuchten sie, wie *Detection Tools* umgangen werden können, die auf maschinellem Lernen basieren. Dazu analysierten sie Angriffe der Kategorie *reverse-mimicry*, wobei Inhalte von schadhaften PDF-Dokumenten in gutartige PDF-Dateien injiziert wurden. Dadurch erhöht sich die Chance, dass eine schadhafte Datei nicht erkannt wird. Unter Anderem wurde auch eingebetteter JavaScript untersucht, jedoch konnte keines der Tools alle bösartigen PDF-Dateien identifizieren.

Auf der letzten „Black Hat“ Konferenz in London, Dezember 2019, haben Abdul-Aziz Hairiri und Edgar Pek eine Möglichkeit vorgestellt, Restriktionen der JavaScript-API im Adobe Acrobat Reader zu umgehen [9]. Die Datei *JSByteCodeWin.bin* wird standardmäßig mit Adobe Acrobat Produkten geliefert und enthält Objekte, Eigenschaften und Funktionen, die von der Anwendung benutzt werden. Darunter befinden sich auch *trusted functions*, welche eingeschränkte JavaScript-Aktionen ausführen können (siehe Abschnitt 4.1.1). Für den Angriff muss eine Möglichkeit gefunden werden, um JavaScript-Code auf Systemebene auszuführen. Danach kann eine Methode innerhalb eines Systemobjekts überschrieben werden, um einen privilegierten API-Aufruf tätigen zu können.

Eine ähnliche Schwachstelle fand der Sicherheitsforscher Alexander Inführer, indem er JavaScript-Code mit Hilfe von vordefinierten, vertrauenswürdigen JavaScript-Funktionen ausführte [13]. Dazu musste seine schadhafte Funktion durch eine vertrauenswürdige Funktion aufgerufen werden.

Valentin Hamon veröffentlichte im Rahmen der „Black Hat“ Konferenz Abu Dhabi 2012 eine Arbeit über bösartige URI-Auflösung in PDF-Dokumenten [8]. Er entdeckte dabei, dass die Prüfung der URL-Aufrufe umgangen werden kann. Durch die JavaScript-Methode `this.submitForm()`; wird ein URL-Aufruf über das Netzwerkverbindugssystem von Adobe Reader (AcroForm) initiiert. Dabei wird die aufgerufene Seite temporär heruntergeladen und auf dem System gespeichert. Anschließend wird die geladene Seite im Standard Browser des Systems geöffnet. Die URL ist in

diesem Fall der Pfad der temporären Datei und nicht die original URL. Falls diese in den Browsereinstellungen gesperrt sein sollte, kann sie dadurch nicht erkannt und geblockt werden.

Die Forscher Frédéric Raynal, Guillaume Delugré und Damien Aumaitre zeigten in ihrer Arbeit einige Schwachstellen im Adobe Reader [19]. Darunter befanden sich auch sicherheitskritische JavaScript-API-Aufrufe, beispielsweise zur Offenlegung von Informationen oder zur Ausführung von Dateianhängen. Diese Aufrufe werden in der vorliegenden Arbeit betrachtet. Zum Schluss stellen sie einen Angriff in Form eines Virus vor, der sich über PDF-Dateien selbst weiter verbreitet, nachdem die Anwendung eines Opfers infiziert wurde. Das PDF-Dokument enthält einen schadhaften Anhang, der die Konfiguration der Anwendung manipuliert. Es wird ein JavaScript-Skript hinzugefügt, das bei jedem Start der Anwendung ausgeführt wird und geöffnete PDF-Dateien infiziert. Diese werden möglicherweise von dem Benutzer weiter geschickt, wodurch sich der Virus ausbreitet.

## 1.3 Struktur der Bachelorarbeit

Die Bachelorarbeit ist in drei Teile aufgeteilt. Im ersten Teil werden die benötigten Grundlagen zu den Themen PDF, JavaScript und Adobe Acrobat Reader DC erläutert.

Der zweite Teil beschäftigt sich mit der *JavaScript for Acrobat API Reference* in der Acrobat DC SDK Dokumentation. Zunächst wird das Angreifermodell und die danach entwickelten Auswahlkriterien für die sicherheitsbedenklichen Methoden und Eigenschaften beschrieben. Diese werden in Kategorien eingeordnet, welche vom Verfasser im Rahmen dieser Arbeit entwickelt und definiert wurden.

Im dritten und letzten Teil sollen die API Calls aus den Kategorien auf ihre Verwendung und Funktionalität getestet werden. Dazu wird ein Testaufbau und die verwendete Methodik eingeführt. Anschließend werden die Ergebnisse ausgewertet. Zuletzt wird das Resultat der gesamten Evaluierung der Sicherheit von JavaScript in PDF-Dateien an dem Beispiel von Adobe Acrobat Reader in einem Fazit festgehalten.



## 2 Grundlagen

In diesem Kapitel werden einige Grundlagen zu den Themen PDFs und JavaScript vermittelt, um die folgende Arbeit verständlicher zu machen.

### 2.1 Begriffserklärungen

Die Arbeit befasst sich mit der Dokumentation der JavaScript API für Adobe Acrobat Produkte, die in der Adobe Acrobat DC SDK Dokumentation festgehalten ist. Dazu werden in den nachfolgenden Abschnitten die wichtigsten Begriffe erläutert, die zum besseren Verständnis beitragen und im Laufe der Arbeit benötigt werden.

#### 2.1.1 SDK - Software Development Kit

Die Abkürzung **SDK** steht für **Software Development Kit** und stellt ein Paket dar, das verschiedene Materialien für die Erstellung von Software für ein bestimmtes Produkt enthält. Typischerweise enthält ein SDK die benötigten APIs, Dokumentationen, zusätzliche Werkzeuge, Bibliotheken und Beispiele. Eine feste Anforderung an ein SDK gibt es allerdings nicht, da es freiwillig von den Entwicklern eines Produkts und meistens frei zugänglich zur Verfügung gestellt wird. Das Acrobat DC SDK ist lizenzfrei und beinhaltet die Dokumentation zu der JavaScript API, auf die diese Arbeit aufbaut.

#### 2.1.2 API - Application Programming Interface

Die Abkürzung **API** steht für **Application Programming Interface** und ist eine Schnittstelle zur Programmierung von Anwendungen. Wörtlich übersetzt bedeutet es Anwendungsprogrammierschnittstelle. Deren Anbindung erfolgt im Gegensatz zu Binärschnittstellen (ABI) auf Quelltextebene.

Durch eine API kann zusätzliche Software für ein Produkt entwickelt werden. Ein berühmtes Beispiel dafür ist die Entwicklung von *Apps* für Smartphones.

In dieser Arbeit geht es um die JavaScript API für Adobe Acrobat Produkte. Durch die Existenz dieser Schnittstelle kann mittels JavaScript mit der Anwendung interagiert werden.

### 2.1.3 API-Aufruf

Ein API-Aufruf initiiert den Prozess der Kommunikation der Anwendung mit der Schnittstelle. In JavaScript besteht ein API-Aufruf aus einem Objekt gefolgt von einer Methode oder Eigenschaft, die für das Objekt verfügbar ist. Die beiden Teile werden durch einen Punkt getrennt.

```
app.alert("Hallo Welt");
```

Listing 2.1: Syntax eines JavaScript API-Aufrufs

## 2.2 PDFs im Allgemeinen

PDF steht für *Portable Document Format* und ist ein von Adobe Inc. entwickeltes Dateiformat für den Austausch von elektronischen Dokumenten. Mit der PDF Version 1.7 wurde das Format 2008 in der ISO 32000-1 [12] spezifiziert und liegt als offener Standard vor. Die aktuelle Version ist 2.0 und wurde in der ISO 32000-2 spezifiziert.

Eine PDF-Datei besteht immer aus vier Teilen: **header**, **body**, **cross-refernce table** und **trailer**.

**header:** Der header ist die erste Zeile in einem PDF-Dokument und beschreibt die Versionsnummer der zugrunde liegenden PDF-Spezifikation. Sie wird auch als *Magic Bytes* bezeichnet und hat das Format %PDF-1.N, wobei N eine Zahl zwischen 0 und 7 sein kann. Mit der aktuellen Version kann auch %PDF-2.0 verwendet werden.

**body:** Der body enthält die indirekten Objekte, aus denen sich die PDF-Datei zusammensetzt, beispielsweise Schriftarten, Textblöcke, Bilder und Metadaten. Ein Objekt hat folgende Syntax: Die Objekte werden durchnummeriert, weswegen die erste Zahl die Objekt Nummer angibt. Danach folgt die Generationsnummer, welche bei nachträglichen Änderungen an einem Objekt inkrementiert wird. Der Inhalt eines Objekts befindet sich zwischen den beiden Schlüsselwörtern **obj** und **endobj**.

```
1 0 obj
/Type /Catalog
/Pages 2 0 R
endobj
```

Listing 2.2: Beispiel eines Catalog-Objekts einer PDF-Datei

Das Catalog-Objekt ist das Wurzelobjekt einer PDF-Datei, welches direkt oder indirekt auf alle anderen Objekte in einem Dokument referenziert. In dem oberen Beispiel wird auf ein *Pages*-Objekt mit der Nummer 2 0 referenziert, welches unter anderem die Anzahl der Seiten des Dokuments angibt.

**cross-reference table:** In einer PDF-Datei ist diese Tabelle unter dem Schlüsselwort *xref* vermerkt und enthält Informationen über die Position der indirekten Objekte im PDF-Dokument. Sie enthält jeweils eine eigene Zeile für jedes indirekte Objekt, die dessen Byte-Offset enthält, um das Objekt schnell lokalisieren zu können.

**trailer:** Der trailer gibt die Position der xref Tabelle und des *Catalog*-Objekts an. Dadurch kann ein PDF-Reader diese Elemente schnell finden und auf sie zugreifen. Normalerweise werden PDF-Dateien von einem Reader vom Ende aus gelesen. Das Ende eines Dokuments wird durch die Markierung **%%EOF** definiert.

## 2.3 JavaScript in PDFs

JavaScript ist eine Skriptsprache, die ursprünglich von der Firma Netscape für den Webbrowser *Navigator 2.0* entwickelt wurde, um die Anwendungsmöglichkeiten von HTML und CSS auszubauen. Damit lassen sich unter anderem dynamische Inhalte verändern und Benutzerinteraktionen auswerten. Die Sprache wurde unter dem Namen *ECMAScript* standardisiert [7].

JavaScript für Acrobat ist eine Erweiterung von JavaScript 1.5 der ISO-16262 und fügt der Hauptsprache neue Objekte mit Methoden und Eigenschaften hinzu. Aktuell wird die JavaScript-Engine von Mozilla, *SpiderMonkey 24*, verwendet. Durch die neuen Objekte kann eine PDF-Datei mit API-Aufrufen in verschiedenen Bereichen manipuliert werden. Dazu zählen beispielsweise Implementierung von Sicherheitsrichtlinien, Interaktionen mit Datenbanken und Webdiensten und dynamisches Verändern von Inhalten eines Dokuments. Der Zugang zu den Standardklassen von JavaScript bleibt aber erhalten.

Die Skripte für PDF-Dateien im Adobe Acrobat Reader DC können auf verschiedene Ebenen platziert werden:

**Ordnerebene:** Diese JavaScript-Skripte werden in einem Ordner auf dem System platziert, der mit der Anwendung verknüpft ist. Sie werden bei der Initialisierung der Anwendung ausgeführt. Die Variablendeklarationen und Funktionen gelten für die gesamte Anwendung und sind für alle Dokumente sichtbar.

**Dokumentebene:** Dokumentebene-Skripte definieren Variablen und Funktionen, die nur für das bestimmte Dokument zugänglich sind. Die Skripte werden in dem Dokument gespeichert.

**Seitenebene:** Diese Skripte werden ausgeführt, wenn eine bestimmte Seite des Dokuments geöffnet oder geschlossen wird und werden ebenfalls im Dokument gespeichert.

**Feldebene:** Diese Skripte sind mit einer Aktion eines bestimmten Formularfeldes verbunden und werden ebenfalls im Dokument gespeichert.

Eine PDF-Datei besteht aus verschiedenen Objekten. Der JavaScript-Code in einer PDF-Datei befindet sich immer in einem **Dictionary** Objekt, welches eine assoziati-

ve Tabelle von Objektpaaren, auch Einträge genannt, enthält. Diese Paare bestehen aus einem **Schlüssel** und einem **Wert** Element, wobei der Schlüssel immer ein Name und der Wert ein beliebiges Objekt ist. Ein JavaScript-Skript wird wie folgt gespeichert:

```
<<
/S /JavaScript
/JS (Skript)
>>
```

Der erste Eintrag gibt den Objekttypen an, in diesem Fall *JavaScript*, und der zweite Eintrag enthält den JavaScript-Code.

## 2.4 JavaScript Events

Alle JavaScript-Skripte werden als Folge eines bestimmten Ereignisses (event) ausgeführt. Einige Ereignisse erzeugen einen privilegierten Kontext, in dem das Skript ausgeführt wird (siehe Abschnitt 4.1.1). Eine JavaScript-Aktion kann gezielt für ein bestimmtes Ereignis gesetzt werden, bei dem es ausgeführt wird.

```
[...]
6 0 obj
<< /JS (app.launchURL\("https://www.nds.ruhr-uni-bochum.de/"\);) /S /JavaScript >>
endobj
[...]
16 0 obj
<< /AA << /O 6 0 R >> /CropBox [ 0.0 0.0 612.0 792.0 ] /MediaBox [ 0.0 0.0 612.0 792.0 ] /
  Parent 11 0 R /Type /Page >>
endobj
[...]
%%EOF
```

Listing 2.3: API-Aufruf für das Ereignis Page/Open

So können API-Aufrufe beispielsweise beim Öffnen oder Schließen des Dokuments oder einer PDF-Seite ausgeführt werden. In dem Beispiel ist ein Objekt mit der Nummer 16 zu sehen, welches das Objekt einer Seite (/Type /Page) in dem Dokument repräsentiert. Der erste Eintrag trägt den Namen **/AA**, welcher ein *additional-actions dictionary* (zusätzliche Aktionen Dictionary-Objekt) spezifiziert. Der Wert des Eintrags (/O) gibt an, dass es sich um ein **Open** Ereignis handelt und referenziert auf das Objekt 6 0. Darin befindet sich die JavaScript-Aktion, die ausgeführt werden soll.

Folgende Ereignisse sind, wie in der Acrobat DC SDK Dokumentation beschrieben, für PDFs verfügbar:

**Kategorie 1 (Doc):** „Doc“ Events interagieren auf Ereignisse, die das Dokument als ganzes Betreffen. Dabei werden Events unterschieden, die vor einem Ereignis („Will“) bzw. nach deren Fertigstellung („Did“) eintreten. Mögliche Ereignisse sind das Öffnen (Doc/Open), Speichern (Doc/DidSave, Doc/WillSave), Schließen (Doc/Close, Doc/WillClose) sowie das Drucken (Doc/WillPrint, Doc/DidPrint).

**Kategorie 2 (Field):** „Field“ Events treten im Zusammenhang mit Formularfeldern ein. Für Interaktionen mit der Maus gibt es ein Ereignis für das Klicken in ein Feld während die Maustaste noch gedrückt ist (Field/Mouse Down), das Klicken in ein Feld und die Maustaste loslassen (Field/Mouse Up), das Bewegen des Mauszeigers in das Feld (Mouse/Enter) und das Verlassen des Mauszeigers aus dem Feld (Field/Exit).

Zusätzlich existiert für Felder jeweils ein Ereignis für den Verlust des Fokus (Field/Blur), das Erhalten des Fokus (Field/Focus), die Erforderlichkeit einer Neuberechnung aller Felder (Field/Calculate), den Abschluss der Neuberechnungen (Field/Format), einen Tastendruck (Field/Keystroke) und die Übergabe einer Benutzereingabe (Field/Validate).

**Kategorie 3 (Page):** „Page“ Events erfolgen bei einer Interaktion mit einer einzelnen Seite. Dafür gibt es das Ereignis für das Betrachten einer neuen Seite (Page/Open) und ein Ereignis für das Verlassen einer Seite (Page/Close). Letzteres kann durch einen Seitenwechsel oder das Schließen des Dokuments ausgelöst werden.

**Kategorie 4 (Screen):** „Screen“ Events beziehen sich immer auf eine *screen annotation* (Bildschirmanmerkung). Das ist ein rechteckiger Bereich innerhalb eines PDF-Dokuments, der auf dem Bildschirm angezeigt wird und mit Wiedergabeversionen und Wiedergabeaktionen für eine Multimedia-Wiedergabe verknüpft sein kann. Dafür existiert ein Ereignis, wenn eine screen annotation zum ersten Mal für einen Benutzer sichtbar wird (Screen/InView). Korrespondierend dazu gibt es Ereignis, welches immer dann eintritt, wenn eine diese Seite zum ersten Mal für den Benutzer nicht mehr sichtbar ist (Screen/OutView).

Für screen annotations gibt es folgende weitere Ereignisse: **Blur**, **Close**, **Focus**, **Mouse Down**, **Mouse Enter**, **Mouse Exit**, **Mouse Up** und **Open**. Sie funktionieren nach dem gleichen Prinzip, wie für die anderen Kategorien beschrieben.

**Kategorie 5 (Sonstiges):** Es existieren zusätzlich noch die alleinstehenden Ereignisse für das Starten der Anwendung (App/Init), das Verarbeiten von Dokumenten in einer Batch-Sequenz, das Anklicken eines Lesezeichens, das eine JavaScript-Aktion ausführt (Bookmark/Mouse Up), das Ausführen von JavaScript-Code in der Konsole (Console/Exec), das Ergebnis eines externen Zugriffs (External/Exec) und das Aktivieren eines Links, der eine JavaScript-Aktion ausführt (Link/Mouse Up).



## 3 Sicherheitskritische API-Aufrufe

Für diesen Teil der Arbeit wurden API-Aufrufe aus der JavaScript für Acrobat API Referenz, die Teil der Acrobat SDK Dokumentation ist, hinsichtlich ihres Sicherheitsrisikofaktors untersucht, bewertet und anschließend in Kategorien eingeteilt. Die Kategorien wurden im Laufe der Arbeit erstellt und werden in den folgenden Abschnitten jeweils erklärt. Es gibt 5 Hauptkategorien mit entsprechender Unterteilung. Zu Beginn jeder Unterkategorie werden alle API-Aufrufe in einer Tabelle aufgeführt. Anschließend werden die wichtigsten API-Aufrufe erläutert.

### 3.1 JavaScript für Acrobat API Referenz

Die JavaScript für Acrobat API Referenz enthält eine umfangreiche Dokumentation der JavaScript API. Die Dokumentation enthält die Adobe Acrobat anwendungsspezifischen Objekte (siehe Abschnitt 2.3), wobei für jedes Objekt die verfügbaren Methoden und Eigenschaften beschrieben sind. Dazu wird zu jedem API-Aufruf eine kleine Tabelle verfasst, die Informationen über Verfügbarkeit und Nutzungsempfehlungen enthält. Einige der API-Aufrufe sind demnach nur mit bestimmten Rechten und in speziellen Situationen verfügbar. Auf diese Eigenschaften wird in dem zweiten Teil der Arbeit detaillierter eingegangen, da vorerst alle API-Aufrufe generell betrachtet werden.

Die Dokumentation bietet des Öfteren anschauliche Beispiele, die die Anwendung und den Nutzen der Methoden und Eigenschaften verdeutlichen.

### 3.2 Wichtige Objekte

In diesem Kapitel werden die wichtigsten Objekte der JavaScript API beschrieben, die bei der Kategorisierung am häufigsten auftauchen. Weitere Objekte werden in den Kategorien kurz erläutert.

**App:** Das App Objekt repräsentiert die Adobe Acrobat Applikation und definiert Acrobat-spezifische Funktionen.

```
app.newDoc();
```

Listing 3.1: API-Aufruf des App Objekts, der ein leeres Dokument erstellt.

**Doc:** Das Doc Objekt ist die primäre Schnittstelle zwischen einem im Viewer geöffneten PDF-Dokument und dem JavaScript-Interpreter. Damit kann auf den Inhalt zugegriffen, um ihn anschließend manipulieren zu können. Das **this** Objekt repräsentiert das aktuell geöffnete Doc-Objekt.

```
this.addField();
```

Listing 3.2: API-Aufruf des Doc Objekts, der im Dokument ein Formularfeld erzeugt.

**Field:** Das Field Objekt steht für ein Acrobat-Formularfeld. Durch die API-Aufrufe des Objekts kann ein Formularfeld modifiziert werden. Bevor auf ein Feld zugegriffen werden kann, muss es über die Methode **this.getField(cName)** an eine JavaScript-Variable gebunden werden. Die verfügbaren Typen für das Field Objekt belaufen sich aktuell (Stand PDF 2.0) auf: *button*, *checkbox*, *combobox*, *listbox*, *radiobutton*, *signature* und *text*.

```
var f = this.getField("fTest");  
f.display = false;
```

Listing 3.3: API-Aufruf des Field Objekts, der das Feld mit dem Namen „fTest“ ausblendet.

**Annotation:** Das Annotation Objekt repräsentiert eine Acrobat-Annotation, die auch *Kommentar* genannt wird. Hier gilt auch: Bevor auf einen Kommentar zugegriffen werden kann, muss es über die Methode **this.getAnnot(cName)** an eine JavaScript-Variable gebunden werden. Die verfügbaren Typen für eine Annotation belaufen sich aktuell (Stand PDF 2.0) auf: *Caret*, *Circle*, *FileAttachment*, *FreeText*, *Highlight*, *Ink*, *Line*, *Polygon*, *PolyLine*, *Redact*, *Sound*, *Square*, *Squiggly*, *Stamp*, *StrikeOut*, *Text* und *Underline*. Eine Liste mit den anwendbaren API-Aufrufen für jeden Typen befindet sich in der JavaScript for API Reference.

```
this.getAnnot("aTest");  
a.contents = "Neuer Text";
```

Listing 3.4: API-Aufruf des Annotation Objekts, der den Inhalt einer Annotation ändert.

### 3.3 Angriffsszenario

Um die Sicherheit eines Systems festzustellen, muss unter anderem klar sein, welche Möglichkeiten ein potentieller Angreifer hat und unter welchen Umständen die Angriffe ausgeführt werden können. Dabei ist es wichtig zu wissen, welche Einstiegs-  
punkte dem Angreifer zur Verfügung stehen.

In dieser Sicherheitsanalyse wird davon ausgegangen, dass ein Angreifer in der Lage ist, beliebige PDF Dateien zu erstellen, in die er beliebigen JavaScript Code implementieren kann. Dabei stehen ihm alle möglichen Events, wie in Abschnitt 2.4 beschrieben, zur Verfügung. Die gewünschte Aktion wird aber erst auf dem System des Opfers ausgeführt.

Die Ausführung einer Aktion beschränkt sich auf die API-Aufrufe innerhalb des Adobe Acrobat Reader DC, wobei es das Ziel eines Angreifers ist, ein Aktion beim Opfer auszuführen, die ohne die Unterstützung von JavaScript in PDF Dateien nicht möglich wäre.

Gegenstand der Arbeit ist es zu ermitteln, welche PDF spezifischen API-Aufrufe aus der JavaScript for API Reference benutzt werden können, um unerwünschte Aktionen auszuführen, die die Sicherheit des Anwenders beeinträchtigen.

### 3.4 Auswahlkriterien

Die Kriterien für die Auswahl sicherheitsbedenklicher API-Aufrufe werden in den folgenden Punkten beschrieben.

Ein API-Aufruf ist sicherheitskritisch, wenn:

- eine Aktion ausgeführt wird, die etwas an einem Dokument oder der Anwendung verändert,
- eine beliebige JavaScript-Aktion gesetzt werden kann,
- auf eine Ressource außerhalb der Anwendung zugegriffen wird,
- Informationen über die Anwendung oder das System offengelegt werden,
- Rechte für eine unerlaubte Aktion erlangt werden.

Diese Kriterien spiegeln die nachfolgenden Kategorien wieder.

## 3.5 Die Kategorien

Es gibt 5 Hauptkategorien, die jeweils einen Themenbereich zusammenfassen. Jede Hauptkatgeorie beinhaltet Unterkategorien, die in diesem Abschnitt erläutert werden. Zu Beginn jeder Unterkategorie wird eine Tabelle mit den zugewiesenen API-Aufrufen angegeben. Die Aufrufe sind in dem Format **Objekt Methode/Eigenschaft** eingetragen. Die API-Aufrufe werden anschließend beschrieben, wobei auch auf einige besondere Parameter eingegangen wird. Eine vollständige Beschreibung der API-Aufrufe ist in der Dokumentation zu finden [10]. Dieser Abschnitt ist in erster Linie dazu da, die Auswahl der Methoden und Eigenschaften vorzustellen und sie aus Sicht der Sicherheitsrelevanz näher zu erläutern.

### 3.5.1 Kategorie 1: Externe Zugriffe

In dieser Kategorie werden API-Aufrufe genannt, die eine Verbindung mit einer Ressource außerhalb der Anwendung aufbauen.

#### 3.5.1.1 URI-Aufrufe

|               |                      |                          |
|---------------|----------------------|--------------------------|
| app launchURL | app.media getURLData | app.media getURLSettings |
| Doc getURL    | Doc submitForm       | app loadPolicyFile       |

Tabelle 3.5: API-Aufrufe der Kategorie: URI-Aufrufe

Die oben aufgeführten API-Aufrufe greifen alle auf eine Ressource außerhalb der Anwendung zu, die durch die Angabe einer URL erreichbar ist. Eine URL kann mit `app.launchURL()` und `this.getURL()` aufgerufen werden, indem sie mit dem Parameter *cURL* an die Methode übergeben wird (siehe Listing 3.6). Beide Methoden öffnen sie automatisch in einem Browser des Systems. Damit könnte ein Angreifer den Anwender auf seine schadhafte Internetseite leiten oder versuchen, eine andere Ressource mit einem anderen Protokoll zu erreichen. Das könnte besonders dann gefährlich werden, wenn der Angreifer zuvor schon Informationen über die Netzwerkstruktur und laufende Dienste des Opfers besitzt. Bei der zweiten Methode ist ein weiterer Parameter, *bAppend*, standardmäßig auf `true` gesetzt und soll dafür sorgen, dass die geöffnete Seite an das Dokument angehängt wird.

```
app.launchURL("https://www.nds.ruhr-uni-bochum.de/");
```

Listing 3.6: URL-Aufruf mit der Methode `app.launchURL`.

Die beiden Methoden `getURLData()` und `getURLSettings()` des **app.media** Objekts beziehen eine Multimedia Datei von einer spezifizierten URL für die Wiedergabe in einem MediaPlayer. Der Unterschied liegt in den verschiedenen Rückgabewerten, führt aber zum gleichen Ziel.

Eine PDF-Datei kann Formularfelder enthalten, die ein Benutzer ausfüllen kann. Mit dem Aufruf von `this.submitForm()` können die Einträge aus den Formularfeldern an eine URL gesendet werden. Für diese Methode gibt es eine Reihe an Parametern, die aber für die Sicherheit nicht von Bedeutung sind, da sie zusätzliche Einstellungen festlegen. Darunter fällt beispielsweise die Auswahl der zu sendenden Felder oder die Wahl eines Formats (FDF, XML, XDP, usw.), in dem die Daten übertragen werden.

Durch die Methode `app.loadPolicyFile()` kann eine Datei geladen werden, die Adobe als Berechtigungsmechanismus verwendet, um PDFs das Laden von Daten von fremden Servern zu erlauben. Das könnte ein Angreifer ausnutzen, um in Kombination mit den ersten beiden API-Aufrufen auf eine verbotene Ressource zuzugreifen.

### 3.5.1.2 Netzwerkdienste

Alle für PDFs verfügbaren Netzwerkdienste kommunizieren über das **Net** Objekt, welches die Erkennung und den Zugriff auf sie über verschiedene Protokolle ermöglicht.

|                              |                             |
|------------------------------|-----------------------------|
| Net.SOAP connect             | NET.SOAP request            |
| NET.SOAP response            | NET.Discovery queryServices |
| NET.Discovery resolveService | NET.HTTP request            |

Tabelle 3.7: API-Aufrufe der Kategorie: Netzwerkdienste

Bis zur aktuellen PDF Version 2.0 werden die Protokolle SOAP und HTTP unterstützt. Außerdem kann über das **Discovery** Objekt dynamisch nach Netzwerkdiensten gesucht werden, indem *DNS Service Discovery* (DNS-SD) [4] verwendet wird. Das ist eine Methode, die Dienste mit Hilfe von Standard-DNS-Abfragen finden kann. Durch die Methode `Net.Discovery.queryServices()` können so Dienste gefunden werden, die sich über *Multicast DNS* (mDNS) auf einer lokalen Netzwerkverbindung oder per *Unicast DNS* innerhalb eines Unternehmens registriert haben. In dem Parameter *cType* kann der *DNS SRV Service Name*, nach dem gesucht werden soll, beispielsweise „http“ oder „ftp“, eingetragen werden. Die Methode `Net.Discovery.resolveService()` ermöglicht es, einen Dienstnamen an eine Netzwerkadresse und einen Port zu binden, damit eine Verbindung hergestellt werden kann. Solche Funktionen können von einem Angreifer genutzt werden, um aus der

Anwendung nach außen zu kommunizieren oder verfügbare Netzwerkdienste zu entdecken.

SOAP ist ein einfaches Protokoll und industrieller Standard des World Wide Web Consortiums (W3C) mit dem Zweck, Daten zwischen zwei Systemen auszutauschen. Die vollständige Spezifikation ist unter [14] zu finden.

In PDF-Dateien wird das Protokoll dafür eingesetzt, um *Remote Procedure Calls* (RPC) zu tätigen oder eine XML Nachricht an einen Server zu versenden. Außerdem kann das **SOAP** Objekt mit Webdiensten kommunizieren, die durch die *Web Services Description Language* [5] beschrieben sind. Ein Angreifer könnte versuchen, eine Verbindung zu einem von ihm kontrollierten Webdienst aufzubauen. Mit der `Net.SOAP.connect()` Methode wird die URL eines WSDL Dokuments zu einem JavaScript Objekt mit aufrufbaren Methoden, die dem Webservice entsprechen, konvertiert. Durch `Net.SOAP.request()` und `Net.SOAP.response()` können anschließend *RPCs* initiiert oder XML-Nachrichten an einen SOAP HTTP Endpunkt geschickt werden.

Mit der `Net.HTTP.request()` Methode lassen sich eigene HTTP-Anfragen gestalten und an eine per **cURL** definierte URL schicken. Durch den Parameter **cVerb** kann ein Parameter aus einem der Nachrichtenprotokolle HTTP, WebDAV und CALDAV gewählt und in **aHeaders** können die dazugehörigen Header gesetzt werden. Weitere Parametereinstellungen sind der Dokumentation zu entnehmen, die allerdings besagt, dass diese Methode nur außerhalb des Kontexts eines Dokuments ausgeführt werden kann.

### 3.5.1.3 Dateisystemzugriffe

Hier werden die Methoden beschrieben, die auf das Dateisystem des Anwenders zugreifen können.

|                         |                       |
|-------------------------|-----------------------|
| Doc saveAs              | global setPersistent  |
| security importFromFile | security exportToFile |

Tabelle 3.8: API-Aufrufe der Kategorie: Dateisystemzugriffe

Die `this.saveAs()` Methode funktioniert wie der gleichnamige Menüpunkt im deutschen UserInterface „Speichern unter“. Jedoch kann hier schon im Voraus mit **cPath** der Pfad angegeben werden, wo das Dokument gespeichert werden soll. Durch die Angabe des Parameters *cConvID* kann das Dokument beim dem Vorgang in ein anderes Format konvertiert werden. Ein Angreifer könnte so versuchen, ein Dokument in einer anderen Form an einer beliebig gewählten Stelle auf dem System zu platzieren.

Mit der Methode `global.setPersistent()` kann eine Variable erstellt werden, auf die dokumentübergreifend zugegriffen werden kann (siehe Listing 3.9). Durch den Parameter `bPersistent = true` kann diese Variable beim Schließen der Anwendung auf dem System in der Datei `globData` gespeichert werden. Nach einem Neustart der Anwendung ist es möglich, auf diese Variable erneut zuzugreifen. Der Standardpfad für diese Datei lautet:

*C:\Users\UserName\AppData\Roaming\Adobe\Acrobat\DC\JSCache.*

```
global.test = 1;
global.setPersistent("test", true);
```

Listing 3.9: Die Variable `test` wird auf den Wert „1“ gesetzt und persistent gespeichert.

Durch die API-Aufrufe `security.exportToFile()` und `security.importFromFile()` kann jeweils ein Zertifikat unter einem gewählten Pfad auf dem System abgespeichert oder eingelesen werden. Damit könnte ein Angreifer versuchen, ein eigenes Zertifikat auf dem System zu platzieren oder ein Zertifikat des Anwenders zu stehlen.

### 3.5.1.4 Programmausführung

Die API-Aufrufe dieser Kategorie führen ein anwendungsexternes Programm aus.

|             |                         |              |
|-------------|-------------------------|--------------|
| app mailMsg | Doc mailDoc             | Doc mailForm |
| Doc print   | PrintParams interactive |              |

Tabelle 3.10: API-Aufrufe der Kategorie: Programmausführung

Die Methoden der ersten Zeile rufen ein Mailprogramm auf dem System auf, während die beiden API-Aufrufe in der zweiten Zeile eine Druckersoftware oder -hardware aufrufen. Sie sind als sicherheitskritisch zu bewerten, da sie alle ohne Benutzerinteraktion ausgeführt werden können. Dafür muss der Parameter `bUI` auf den Wert `false` gesetzt werden, wodurch kein UserInterface angezeigt wird. Der Parameter `bSilent = true` sorgt bei der Methode zum Drucken zusätzlich dafür, dass keine Dialogbox mit der Option, den Vorgang abubrechen, angezeigt wird. `interactive` ist eine Eigenschaft des `PrintParams` Objekts, welcher an die `print`-Funktion übergeben werden kann. Diese Eigenschaft kann ebenfalls auf `silent` gesetzt werden, was den gleichen Effekt wie `bSilent = true` bewirkt.

### 3.5.1.5 Ausführung einer beliebigen Datei mit Dateisystemzugriff

Eine Möglichkeit, angehängte Dateien auf das System des Anwenders zu exportieren und auszuführen, bietet die Methode `this.exportDataObject()`. Das Spei-

|                      |
|----------------------|
| Doc exportDataObject |
|----------------------|

Tabelle 3.11: API-Aufrufe der Kategorie: Ausführung einer beliebigen Datei mit Dateisystemzugriff

chern eines Anhangs erfordert immer eine Benutzerinteraktion, womit der Speicherort ausgewählt wird. Das Gefährliche an dieser Methode ist der Parameter **nLaunch**. Ist dieser Wert gleich „2“, wird die Datei temporär gespeichert und sofort geöffnet. Damit kann ein Angreifer eine beliebige Datei auf dem System ausführen.

|                                                                        |
|------------------------------------------------------------------------|
| <code>this.exportDataObject({ cName: "test.txt", nLaunch: 2 });</code> |
|------------------------------------------------------------------------|

Listing 3.12: Die Methode speichert die Datei „test.txt“ temporär und öffnet sie.

### 3.5.2 Kategorie 2: Anwendungsinteraktionen

Alle API-Aufrufe dieser Kategorie interagieren mit der Anwendung Adobe Acrobat Reader DC, indem sie Aussehen oder Funktionen manipulieren.

#### 3.5.2.1 UI-Manipulation

Die folgenden Aufrufe manipulieren das User-Interface (UI) der Anwendung. Dabei werden sichtbare Elemente der Anwendung hinzugefügt oder entfernt.

|                   |                  |
|-------------------|------------------|
| app addItem       | app addSubMenu   |
| app addToolButton | app hideMenuItem |

Tabelle 3.13: API-Aufrufe der Kategorie: UI-Manipulation

Mit `app.addItem()` kann einem Menü ein neuer Menüeintrag hinzugefügt werden. Der Name und die Position ist frei wählbar und kann durch die entsprechenden Parameter festgelegt werden. Außerdem kann in dem Parameter **cExec** ein String eingetragen werden, der bei der Auswahl des Eintrags ausgewertet wird. Damit kann auch beliebiger JavaScript-Code eingetragen werden. In dem Beispiel wird dem Menü **Datei** der Menüeintrag „Hallo“ hinzugefügt (siehe Listing 3.14). Dieser öffnet ein Dialogfenster mit dem Inhalt „Hallo“.

```
app.addItem({
  cName: "Hallo",
  cParent: "File",
  cExec: "app.alert(\"Hallo\");",
  nPos: 0
});
```

Listing 3.14: Die Methode `app.addItem()` fügt einen neuen Menüeintrag hinzu.

Durch `app.addSubMenu()` können zusätzlich noch Untermenüeinträge erstellt werden. Die Methode `app.hideMenuItem()` entfernt einen Menüeintrag einschließlich bereits existierende Einträge. Damit könnte eine Angreifer einen Standardmenüeintrag entfernen und durch einen gleichnamigen ersetzen, um den Benutzer eine andere Aktion ausführen zu lassen.

Ein **ToolButton** wird in der Anwendung auf der rechten Seite in der Werkzeugleiste angezeigt und kann ebenfalls beliebigen JavaScript-Code ausführen. Falls er in einem nicht-privilegierten Kontext durch JavaScript erstellt wurde, wird er mit einer Warnung gekennzeichnet. Neue Toolbuttons sind aufgrund ihrer Position und der Warnung leicht zu identifizieren und stellen eine weniger große Gefahr dar.

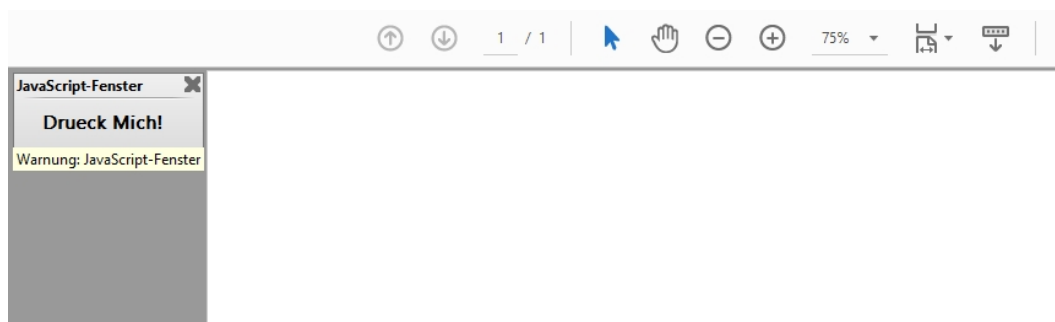


Abbildung 3.15: Toolbutton mit Warnung: JavaScript-Fenster

### 3.5.2.2 Anwendungseigenschaftenmanipulation

|                                   |                                    |
|-----------------------------------|------------------------------------|
| security validateSignaturesOnOpen | SecurityHandler setPasswordTimeout |
|-----------------------------------|------------------------------------|

Tabelle 3.16: API-Aufrufe der Kategorie: Anwendungseigenschaftenmanipulation

Diese beiden Aufrufe können eine Einstellung, die für die ganze Anwendung gilt, manipulieren. Durch das Setzen des Wertes von `security.validateSignaturesOnOpen` auf **false** kann verhindert werden, dass Signaturen beim Öffnen des Dokuments automatisch validiert werden. Dann müssen Signaturen manuell geprüft werden, indem

in der linken Leiste das Unterschriften-Symbol ausgewählt wird. Dort kann die Aktion „Alle prüfen“ ausgeführt werden, die alle Unterschriften in einem Dokument überprüft. Die Eigenschaft gehört zu dem **security** Objekt, welches sicherheitsrelevante PDF-Funktionen wie Verschlüsselung und digitale Signaturen bereitstellt.

Bevor eine Signatur einem Dokument hinzugefügt werden kann, muss immer das entsprechende Passwort für das Zertifikat eingegeben werden. Der zweite Aufruf ist eine Methode, mit der die Anzahl der Sekunden festgelegt werden kann, nach denen das Passwort für die Signierung eines Dokuments erneut angefordert werden soll. Danach muss ein Benutzer sein Passwort erneut eingeben. Die Einsatzmöglichkeit der Methode ist dadurch eingeschränkt, dass sie nur vom Standard security handler *Adobe.PPKLite* unterstützt wird. Ein security handler stellt einen Mechanismus, um einen Benutzer zu authentifizieren. In Adobe Produkten sind das Plugins, die unter Anderem zum Ausführen von privaten Schlüsselfunktionen, zur Validierung von Signaturen und zum Signieren und Verschlüsseln von Dokumenten verwendet werden [11]. Es können auch eigene security handler in der Anwendung installiert und benutzt werden. Ein gut informierter Angreifer könnte sie vielleicht trotzdem ausnutzen.

### 3.5.2.3 UI-Interaktion

Der Reader kann diverse Dialogfenster erzeugen, die mit dem Benutzer interagieren.

|              |                 |
|--------------|-----------------|
| app alert    | app execDialog  |
| app response | app popUpMenuEx |

Tabelle 3.17: API-Aufrufe der Kategorie: UI-Interaktion

Ein Warnungsdialogfenster mit einer selbst gewählten Nachricht kann durch `app.alert()` erzeugt werden. `app.response()` erzeugt ein Dialogfenster erzeugt, das dem Benutzer eine Frage stellt und ein Textfeld für die Antwort bereitstellt. Das Textfeld kann bereits mit einem Standardwert befüllt werden. Ein weitaus komplexeres Dialogfenster kann durch den API-Aufruf `app.execDialog()` produziert werden. Mit Hilfe dieser Methode können ganze Klickstrecken, eine Reihe an aufeinander folgenden Dialogfenstern, erzeugt werden, wobei mehrere Eingaben des Benutzers gespeichert werden können. Durch ein geschicktes Gestalten einer Dialogbox könnte ein Angreifer den Benutzer dazu verleiten, sensible Informationen preiszugeben. Der Aufruf von `app.popUpMenuEx()` lässt ein PopUp-Menü mit vorher ausgewählten Menüeinträgen erscheinen. Die Aktion des Eintrags wird bei der Auswahl ausgeführt.

### 3.5.2.4 Ausführung einer Anwendungsaktion

|            |              |                  |
|------------|--------------|------------------|
| app newDoc | Doc closeDoc | app execMenuItem |
|------------|--------------|------------------|

Tabelle 3.18: API-Aufrufe der Kategorie: Ausführung einer Anwendungsaktion

Mit den API-Aufrufen dieser Kategorie lassen sich einige Anwendungsaktionen ausführen, beispielsweise die Erstellung eines neuen Dokuments. Die `this.closeDoc()` Methode schließt ein Dokument und unterdrückt die Speicherung von geänderten Inhalten, wenn der Parameter `bNoSave` auf den Wert `true` gesetzt ist.

Ein beliebiger Menüeintrag kann durch den Aufruf von `app.execMenuItem()` ausgeführt werden. Jedoch betrifft dies nicht alle Menüeinträge, da manche nur in einem privilegierten Kontext ausgeführt werden können. Eine Liste der allgemein erlaubten Einträge befindet sich in der Dokumentation von Adobe.

### 3.5.2.5 Automatisierte Codeausführung

|                 |                |
|-----------------|----------------|
| app setInterval | app setTimeout |
|-----------------|----------------|

Tabelle 3.19: API-Aufrufe der Kategorie: Automatisierte Codeausführung

Beide Methoden dieser Unterkategorie führen einen beliebigen JavaScript-Code zu einem gewünschten Zeitpunkt aus. Die erste Methode führt diesen jedoch öfters in einem vorher festgelegten Intervall aus, während die zweite Methode den Code nur einmal ausführt.

## 3.5.3 Kategorie 3: Offenlegung von Informationen

In dieser Kategorie befinden sich alle API-Aufrufe, die ein Angreifer ausnutzen könnte, um an Informationen über die Anwendung oder das unterliegende System zu kommen.

|                            |                      |                           |
|----------------------------|----------------------|---------------------------|
| app formsVersion           | app platform         | app plugIns               |
| app printerNames           | app viewerType       | app viewerVariation       |
| app viewerVersion          | app.media version    | app.media getPlayers      |
| Doc path                   | app getPath          | app listMenuItems         |
| DirConnection search       | identity corporation | identity email            |
| identity loginName         | identity name        | SecurityHandler loginPath |
| SecurityHandler digitalIDs |                      |                           |

Tabelle 3.20: API-Aufrufe der Kategorie: Offenlegung von Informationen

Durch die Offenlegung von Versionsnummern durch die Eigenschaften `app.formsVersion`, `app.viewerVersion` und `app.media.version` können Schwachstellen identifiziert werden, z.B über CVE Einträge [1].

Die Eigenschaft `app.printerNames` gibt eine Liste mit den verfügbaren Druckern auf dem System aus und die Methode `app.media.getPlayers()` listet die verwendbaren MediaPlayer auf. Die Informationen können einem Angreifer mögliche Schwachstellen offenbaren. Über den Aufruf `app.pluginIns` wird eine Liste mit den in der Anwendung installierten Plug-ins ausgegeben. Es gibt beispielsweise ein Plug-In zur Prüfung der Rechtschreibung (Spelling) und für die Sprachausgabe (ReadOutLoud). Ein Angreifer könnte diese Informationen ebenfalls verwenden, um bereits bekannte Schwachstellen zu identifizieren<sup>1</sup>.

Des Weiteren können Informationen über das Dateisystem erlangt werden, indem `this.path` den geräteunabhängigen Pfad des Dokuments ausgibt und `app.getPath()` die Pfade zu den Ordnern anzeigt, die bei der Installation erstellt wurden.

Das **DirConnection** Objekt repräsentiert eine offene Verbindung zu einem Datenverzeichnis (Directory). Diese Datenverzeichnisse enthalten Benutzerinformationen inklusive Public-Key Zertifikate. Das Standardverzeichnis des Readers ist *Adobe.AAB.AAB* (Acrobat Address Book) und kann über den security handler *Adobe.AAB* erreicht werden. Durch die Methode `search` können die eingetragenen Benutzerentitäten ausgelesen werden. Mit der Eigenschaft `digitalIDs` eines **SecurityHandler** Objekts können auch die Zertifikate der digitalenIDs des jeweiligen security handlers ausgegeben werden. Damit könnte ein Angreifer an sensible Daten gelangen.

Die übrigen Aufrufe geben noch weitere Informationen über die Anwendung aus, wie zum Beispiel die Versionsnummer.

### 3.5.4 Kategorie 4: Dokumentmanipulation

In dieser Kategorie geht es um API-Aufrufe, die direkt auf ein Dokument zugreifen. Viele dieser Aufrufe verändern Teile des Inhalts oder Eigenschaften eines Dokuments nachträglich, welche beim Schließen gespeichert werden. Damit können Inhalte geändert und Benutzereingaben unerlaubt verfälscht werden. Manipulationen an Formularfeldern und Annotationen verändern zwar auch den Dokumentinhalt, werden aber in einzelnen Unterkategorien behandelt, um eine besser Übersicht zu geben.

---

<sup>1</sup><https://www.fortinet.com/blog/threat-research/fortinet-researchers-discover-two-critical-vulnerabilities-in-adobe-acrobat-and-reader.html>

### 3.5.4.1 Dokumenteigenschaftenmanipulation

Durch JavaScript können einige Eigenschaften eines Dokuments verändert werden.

|                   |                          |               |
|-------------------|--------------------------|---------------|
| Doc baseURL       | Doc disclosed            | Doc info      |
| Doc addScript     | Doc certifyInvisibleSign | Doc setAction |
| Doc setPageAction |                          |               |

Tabelle 3.21: API-Aufrufe der Kategorie: Dokumenteigenschaftenmanipulation

Die **baseURL** wird in einem Dokument genutzt, um relative Web Links aufzulösen. Durch den entsprechenden API-Aufruf kann sie geändert werden. Das könnte ein Angreifer ausnutzen, um den Benutzer beim Aufrufen eines Links auf eine andere Seite zu leiten, ohne dass dieser es bemerkt.

Eine weitere potentiell gefährliche Eigenschaft eines Dokuments ist die **this.disclosed** Eigenschaft. Wenn sie auf den Wert **true** gesetzt wird, ist das Dokument für JavaScript-Skripte aus anderen Dokumenten zugänglich. Ein Angreifer könnte versuchen, diese Eigenschaft für andere Dokumente zu setzen, um sie mit JavaScript zu manipulieren. Das **info** Objekt eines Dokuments enthält unter anderem Informationen über Titel, Autor, Erstellungsdatum und Datum der letzten Veränderung, die ein Angreifer nachträglich verändern könnte.

Mit den Methoden **this.addScript()**, **this.setAction()** und **this.setPageAction()** können einem Dokument JavaScript-Aktionen zugewiesen werden, die in verschiedenen Situationen auslösen. Die erstgenannte Methode setzt ein Skript, welches beim Öffnen des Dokuments ausgelöst wird, und die letzte führt das Skript beim Öffnen einer bestimmten Seite aus. Für **this.setAction()** kann eins der folgenden Ereignisse (siehe Abschnitt 2.4) gewählt werden, bei denen die JavaScript-Aktion ausgelöst wird: *WillClose*, *WillSave*, *DidSave*, *WillPrint*, *DidPrint*. In dem folgenden Beispiel wird ein Skript gesetzt, das beim Schließen des Dokuments einen Ton abspielt.

```
var script = "app.beep(0);";
this.setAction("WillClose", script);
```

Listing 3.22: Beispiel zur Methode **this.setAction**.

Durch die Methode **this.certifyInvisibleSign()** kann ein Dokument mit einem unsichtbaren Zertifikat zertifiziert werden. Ein Angreifer könnte versuchen, ein Dokument mit einem Zertifikat des Anwenders zu zertifizieren, wozu jedoch ein security handler benötigt wird. Dieser könnte einen Benutzernamen und ein Passwort fordern, weswegen der Versuch wahrscheinlich misslingen wird.

### 3.5.4.2 Dokumentinhaltmanipulation

Mithilfe von JavaScript, können auch die Inhalte eines Dokuments verändert werden. Dabei wird auf die Datenobjekte zugegriffen, aus denen ein Dokument besteht (siehe Abschnitt 2.2). Ein Angreifer könnte versuchen, die Inhalte eines Dokuments somit nachträglich zu verändern.

|                                 |                           |
|---------------------------------|---------------------------|
| Doc deletePages                 | Template spawn            |
| Doc createDataObject            | Doc setDataObjectContents |
| Doc addRecipientListCryptFilter | Doc applyRedactions       |
| Link setAction                  | ocg initState             |
| ocg setAction                   | Doc addLink               |
| Doc addField                    | Doc addAnnot              |
| Doc addWatermarkFromText        | ocg locked                |

Tabelle 3.23: API-Aufrufe der Kategorie: Dokumentinhaltmanipulation

Mit der `this.deletePages()` Methode können Seiten eines Dokuments entfernt werden. Dabei ist es nicht möglich, alle Seiten eines Dokuments zu löschen.

Eine Seite eines Dokuments kann mit dem Code `var T = this.createTemplate()` als Vorlage gespeichert und mit `T.spawn()` als exakte Kopie hinzugefügt werden. Damit könnte eine Seite aus dem aktuellen Dokument in ein anderes Dokument übernommen werden.

Ein neues Datenobjekt lässt sich mit dem API-Aufruf `this.createDataObject()` ad-hoc erstellen und kann durch `this.setDataObjectContents()` mit beliebigem Inhalt befüllt werden. Diese Methode kann auch auf bestehende Datenobjekte angewendet werden, um den Inhalt zu verändern. Dabei wird der Inhalt als Datenstrom in einem *ReadStream* Objekt übergeben. Mithilfe der Methoden des **util** Objekts kann ein Datenstrom in das gewünschte Format konvertiert werden.

Durch *Annotationen* können konkrete Vorschläge für eine Änderung an einem Text in einer PDF-Datei gemacht werden. Sie sind vom Typ *Redaction* und enthalten einen alternativen Text und markieren die zu ändernde Stelle. Durch den Aufruf der Methode `this.applyRedactions()` wird der Text an der markierten Stelle durch den alternativen Text ersetzt. Das könnte eine Angreifer ausnutzen, um beliebigen Textinhalt in einer PDF-Datei zu verändern.

In einer PDF-Datei gibt es *optionale Inhalte*, die einen bestimmten Inhalt beschreiben, welcher selektiv angezeigt oder ausgeblendet werden kann [12]. Dies ist eine nützliche Funktion, um beispielsweise CAD-Zeichnungen oder Wasserzeichen in einem Dokument zu verwalten. Solche Inhalte werden in einer **Optional Content Group (OCG)** gespeichert, die eigene Objekte in einer PDF-Datei sind. Das Objekt kann durch JavaScript manipuliert werden, wodurch der Status der Sichtbarkeit eines Inhalts mit `initState` auf **on** oder **off** festgelegt werden kann. Der Status kann

mit `locked` gesperrt werden, wodurch er sich nicht mehr durch das User-Interface ändern lässt. Es kann auch ein JavaScript-Ausdruck mit `setAction()` gesetzt werden, der bei einer Änderung des Status ausgeführt wird. Durch den Aufruf von `this.addWatermarkFromText()` kann einem Dokument ein Wasserzeichen in Form eines Textes als OCG hinzugefügt werden. Dieses könnte von einem Angreifer nachträglich sichtbar gemacht oder ausgeblendet werden.

Weitere Elemente können einem Dokument durch die Methoden `this.addField()`, `this.addAnnot()` und `this.addLink()` hinzugefügt werden. Die Erklärung zu den ersten beiden Elementen erfolgt in den nachfolgenden Abschnitten. `this.addLink()` platziert einen Link im Dokument auf einer Seite und an der Stelle der eingetragenen Koordinaten. Der Link kann durch die Methode `setAction()` des **Link** Objekts mit einer JavaScript-Aktion versehen werden, die bei der Aktivierung ausgeführt wird.

### 3.5.4.3 Dokumentinhaltmanipulation mit Dateisystemzugriff

Einer PDF-Datei können auch Inhalte von einer Quelle außerhalb der Anwendung hinzugefügt werden. Dafür wird auf das unterliegende Dateisystem zugegriffen.

|                          |                      |
|--------------------------|----------------------|
| Doc insertPages          | Doc replacePages     |
| Doc addWatermarkFromFile | Doc importDataObject |

Tabelle 3.24: API-Aufrufe der Kategorie: Dokumentinhaltmanipulation mit Dateisystemzugriff

Ein Wasserzeichen kann mit der Methode `this.addWatermarkFromFile()` als eigenständige Seite durch die Angaben eines Pfades aus einer externen Datei erzeugt und den Seiten eines Dokuments hinzugefügt werden. Dabei wird es wieder in einer OCG abgespeichert. Falls die externe Datei keine PDF-Datei ist, versucht die Anwendung den gewählten Inhalt in eine PDF zu konvertieren.

Die Methoden `this.insertPages()` und `this.replacePages()` fügen die gewählten Seiten aus einem PDF-Dokument, das sich unter dem angegebenen Pfad befindet, der Ziel-PDF-Datei hinzu bzw. ersetzen sie.

Mit `this.importDataObject()` kann eine externe Datei durch die gleiche Vorgehensweise als Datenobjekt in eine PDF-Datei importiert werden. Durch den Parameter **cName** wird ein Name für das Objekt festgelegt.

### 3.5.4.4 Feldmanipulation

Die API-Aufrufe in dieser Kategorie dienen der Manipulationen von Formularfeldinhalten und -eigenschaften. Falls diese nachträglich geändert werden können, stellt das ein Sicherheitsrisiko dar, da somit Inhalte und vor allem Benutzereingaben verfälscht werden können.

|                                        |                                          |                                  |
|----------------------------------------|------------------------------------------|----------------------------------|
| Field <code>currentValueIndices</code> | Field <code>defaultValue</code>          | Field <code>display</code>       |
| Field <code>password</code>            | Field <code>value</code>                 | Field <code>checkThisBox</code>  |
| Field <code>deleteItemAt</code>        | Field <code>insertItemAt</code>          | Field <code>setItems</code>      |
| Field <code>exportValues</code>        | Field <code>defaultIsChecked</code>      | Field <code>setAction</code>     |
| Field <code>setLock</code>             | Field <code>signatureSetSeedValue</code> | Field <code>signatureSign</code> |

Tabelle 3.25: API-Aufrufe der Kategorie: Feldmanipulation

Es gibt verschiedene Arten von Formularfeldern, die eine PDF-Datei enthalten kann (siehe Abschnitt 3.2). Eine Listbox oder Combobox enthält Einträge, die mit JavaScript manipuliert werden können. Durch die API-Aufrufe `f.deleteItemAt()` und `f.insertItemAt()` können Einträge an einer bestimmten Position gelöscht oder hinzugefügt werden. Mit der Methode `f.setItems()` lässt sich gleich eine ganze voreingestellte Liste auf einmal einsetzen. Die Einträge in einer Liste besitzen jeweils zusätzlich zu dem Anzeigenamen auch einen Exportwert, welcher bei der Auswahl weiterverarbeitet wird. So kann der Anzeigename beispielsweise *Annehmen* lauten und der dazugehörige Exportwert *1* sein. Exportwerte lassen sich mit JavaScript durch den Aufruf `f.exportValues` ändern und können auch bei Checkboxes oder Radiobuttons dadurch verändert werden. Der Status von Checkboxes und Radiobuttons kann mit der Methode `f.checkThisBox()` auf aktiviert oder deaktiviert gesetzt werden. Die Methode `f.defaultIsChecked()` setzt bei den beiden die Standardeinstellung dazu.

Die Auswahl von Einträgen in einer Liste ist der Wert dieser Liste und kann durch den Aufruf `f.currentValueIndices` manuell gesetzt werden. Dabei ist auch eine Mehrfachauswahl möglich, wenn die dafür benötigte Eigenschaft für das Feld erfüllt ist. Der Wert wird demnach als Integer oder Array gesetzt, wobei die Zählung der Listeneinträge bei null beginnt.

Der Wert eines Feldes hängt vom Typ ab und kann immer mit dem API-Aufruf `f.value` abgerufen und verändert werden. Für ein Textfeld ist der Wert der eingetragene Text und damit meistens eine Benutzereingabe. Textfelder besitzen zudem die Eigenschaft `f.password`, welche eine Benutzereingabe mit Asterisk-Symbolen maskiert und sie beim Schließen eines Dokuments nicht speichert. Die Eigenschaft kann mit JavaScript auf `true` oder `false` gesetzt werden. Das könnte ein Angreifer ausnutzen, um diese Eigenschaft vor dem Schließen noch zu ändern, sodass ein eingegebenes Passwort doch noch gespeichert wird.

Die Eigenschaft `f.display` gilt für alle Formularfeldtypen und bestimmt, ob ein Feld ein- oder ausgeblendet wird.

Auch für ein Feld kann eine JavaScript-Aktion für einen bestimmten Auslöser durch die Methode `f.setAction()` festgelegt werden. Meistens wird dadurch eine Aktion für einen Button gesetzt.

Felder können durch den API-Aufruf `f.setLock()` gesperrt werden, wodurch sie nach einem Signiervorgang nicht mehr verändert werden können.

Durch die Methoden `f.signatureSetSeedValue()` und `f.signatureSign()` können zuerst Voreinstellungen für die Signatur eines Feldes gesetzt werden, welche anschließend auf ausgewählte Felder angewendet werden. Damit könnte ein Angreifer wieder versuchen, die Signatur des Benutzers zu verwenden um ein Feld zu signieren. Die Einstellungen dafür sind jedoch ziemlich komplex und umfangreich, weswegen zunächst nur getestet wird, ob diese Methoden überhaupt aufgerufen werden können.

#### 3.5.4.5 Annotationmanipulation

Annotationen liegen zwar von der Logik her eine Schicht über dem eigentlichen Inhalt einer PDF-Datei, gehören aber trotzdem zu einem Dokument dazu und können auch per JavaScript Aufruf manipuliert werden.

|                              |                      |                     |
|------------------------------|----------------------|---------------------|
| Annotation author            | Annotation contents  | Annotation hidden   |
| Annotation lock              | Annotation modDate   | Annotation readOnly |
| Annotation destroy           | Annotation setProps  | Annotation state    |
| Annotation transitionToState | Annotation inReplyTo | Annotation type     |
| Collab addStateModel         |                      |                     |

Tabelle 3.26: API-Aufrufe der Kategorie: Annotationmanipulation

Eine Annotation enthält mehrere Eigenschaften, die manipuliert werden können. Dazu zählt auch der Name des Verfassers eines Kommentars, der unter der Eigenschaft `a.author` gespeichert ist und über einen API-Aufruf geändert werden kann. Wie schon bei den Formularfeldern kann auch hier die Sichtbarkeit geregelt werden, indem `a.hidden` auf `true` oder `false` gesetzt wird. `a.lock = true` sperrt einen Kommentar, was aber in diesem Kontext mit der Eigenschaft `a.readOnly = true` gleichzusetzen ist, außer dass die Annotation über das Eigenschaften-Dialogfeld in der Benutzeroberfläche zugänglich ist.

Annotationen gibt es in verschiedenen Versionen, die jeweils in der `a.type` Eigenschaft festgelegt ist. Durch eine Änderung des Typen kann das Aussehen und die Funktion einer Annotation modifiziert werden. Außerdem kann einer Annotation ein Status zugewiesen werden, der aus einem Statusmodell ausgewählt wird. Das Standardstatusmodell beinhaltet die Auswahlmöglichkeiten *Kein*, *Akzeptiert*, *Abgebrochen*, *Beendet* und *Abgelehnt*. Durch die Methode `Collab.addStateModel()` des Collab-Objekts kann ein neues Statusmodell mit eigenen Einträgen erstellt werden. Der Status kann mit dem API-Aufruf `a.state` gesetzt oder geändert werden. Eine weitere Möglichkeit bietet die Methode `a.transitionToState()`.

Die Eigenschaft `a.inReplyTo` enthält den Namen des Kommentars, auf den dieser Kommentar antwortet. Durch die Möglichkeit, diese Eigenschaften mit JavaScript zu

setzen, könnte ein Angreifer beliebig Kommentare nachträglich verändern. So könnte er sogar Kommentarverläufe nachstellen, indem er neue Kommentare hinzufügt und diese dann als Antwort auf andere Kommentare setzt. Dazu kommt auch noch, dass die Eigenschaft `a.modDate` eines Kommentars verändert werden kann. Sie gibt das Datum der letzten Änderung an der Annotation an. Weiterhin kann der Inhalt einer Annotation mit dem API-Aufruf `a.contents` geändert werden. In dem folgenden Beispiel wird der Inhalt und das Datum der letzten Änderung der Annotation „test“ geändert.

```
var a = this.getAnnot("test");
a.contents = "Neuer Text";
a.modDate = "D:20000801145605+07'00'";
```

Listing 3.27: Beispiel zu den API-Aufrufen `a.modDate` und `a.contents`.

Mit der Methode `a.setProps()` lassen sich mehrere Eigenschaften auf einmal setzen und `a.destroy()` entfernt eine Annotation vollständig.

#### 3.5.4.6 Annotationmanipulation mit Dateisystemzugriff

|                            |
|----------------------------|
| Annotation cAttachmentPath |
|----------------------------|

Tabelle 3.28: API-Aufrufe der Kategorie: Annotationmanipulation mit Dateisystemzugriff

Wenn eine Annotation vom Typ *FileAttachment* ist, kann ihr eine Datei angehängt werden. Den Pfad zu der Datei, die angehängt werden soll, kann durch den Parameter `cAttachment` angegeben werden. Ein Angreifer könnte dadurch versuchen, eine sensible Datei vom System an einen Kommentar zu hängen und die PDF-Datei vom Benutzer zurückzufordern, um an die Datei zu gelangen.

#### 3.5.4.7 Verzögerte Ausführungen

|           |                  |             |
|-----------|------------------|-------------|
| Doc delay | Annotation delay | Field delay |
|-----------|------------------|-------------|

Tabelle 3.29: API-Aufrufe der Kategorie: Verzögerte Ausführungen

Die `this.delay` Eigenschaft existiert für die drei in Tabelle 3.29 aufgeführten Objekte und kann die Anwendung von Änderungen verzögern. Dafür muss der Wert der Eigenschaft zuerst auf `true` gesetzt werden, damit nachfolgende Änderungen nicht direkt übernommen werden. Für einen auf `false` gesetzten Wert, werden alle Änderungen auf einmal ausgeführt. Das **Doc** Objekt bezieht sich dabei auf sämtliche Änderungen an allen Formularfeldern in einem Dokument, während mit `f.delay`

einzelne Felder ausgesucht werden können. `a.delay` verzögert Änderungen an einzelnen Kommentaren.

Diese Funktion könnte ausgenutzt werden, um Änderungen bei einer gewählten Aktion auszuführen, sodass der Benutzer es nicht mitbekommt.

### 3.5.4.8 Bookmarks

Eine PDF-Datei kann Lesezeichen enthalten, die üblicherweise an der linken Seite des Readers zu sehen sind.

|                      |                  |                    |
|----------------------|------------------|--------------------|
| Bookmark createChild | Bookmark execute | Bookmark setAction |
|----------------------|------------------|--------------------|

Tabelle 3.30: API-Aufrufe der Kategorie: Bookmarks

Mit der ersten Methode aus der überliegenden Tabelle kann ein Lesezeichen mit einer ausführbaren JavaScript-Aktion an einer beliebigen Stelle im Lesezeichenbaum erstellt werden. Die Methode `setAction()` kann einem Lesezeichen eine Aktion nachträglich zuweisen oder eine bestehende Aktion überschreiben. Durch den Aufruf von `execute()` wird die JavaScript-Aktion eines ausgewählten Lesezeichens manuell ausgeführt. Dies bietet einem Angreifer eine weitere Möglichkeit, beliebigen JavaScript-Code ausführen zu lassen.

### 3.5.4.9 Zugriff auf andere Dokumente

|                |
|----------------|
| app activeDocs |
|----------------|

Tabelle 3.31: API-Aufrufe der Kategorie: Zugriff auf andere Dokumente

Durch den Aufruf dieser Eigenschaft wird ein Array mit den Doc-Objekten für jedes offene Dokument in der Anwendung ausgegeben. Dadurch kann ein Angreifer versuchen, andere geöffnete Dokumente zu manipulieren. Um die API-Aufrufe des Doc Objekts auf das eigene Dokument anzuwenden, muss `this.apiAufruf` verwendet werden. Falls auf ein anders offenes Dokument zugegriffen werden soll, wird mit `var a = app.activeDocs` das Array mit den offenen Dokumenten in einer Variable gespeichert. Danach kann durch `a[0].apiAufruf` mit dem ersten offenen Dokument, falls vorhanden, interagiert werden.

### 3.5.4.10 Temporäres Abspeichern eines Dokumentinhalts

Mit den folgenden API-Aufrufen können einige Inhalte eines Dokuments in einer Variable zwischengespeichert werden, um sie nachher weiterzuverarbeiten.

|                               |                           |
|-------------------------------|---------------------------|
| Collab documentToStream       | Doc getDataObjectContents |
| annotAttachment contentStream |                           |

Tabelle 3.32: API-Aufrufe der Kategorie: Temporäres Abspeichern eines Dokumentinhalts

Durch die `Collab.documentToStream()` Methode kann eine Kopie eines Dokuments als `ReadStream` Objekt gespeichert werden. Die anderen beiden Methoden speichern jeweils ein Datenobjekt oder den Inhalt eines Anhangs ebenfalls in einem `ReadStream` Objekt ab. Ein Angreifer könnte damit versuchen, an Inhalte aus einem anderen Dokument zu gelangen.

### 3.5.5 Kategorie 5: Rechteausweitung

Einige der Funktionen, die in diesem Kapitel beschrieben wurden, können aus Sicherheitsgründen nur in einem privilegierten Kontext ausgeführt werden. Die Methode `app.trustedFunction()` markiert eine Funktion als vertrauenswürdig und erhöht das Privilegienlevel für ihren Inhalt. Im Body der Methode muss jeglicher Code, der einen privilegierten Kontext erfordert, zwischen den Methoden `app.beginPriv()` und `app.endPriv()` stehen, um mit erhöhten Rechten ausgeführt zu werden.

## 4 Testaufbau

Im ersten Teil der Arbeit wurden potentiell sicherheitskritische API-Aufrufe aus der JavaScript API ermittelt und in Kategorien aufgeteilt, um eine bessere Übersicht zu schaffen. Nun sollen diese API-Aufrufe auf ihre Einsetzbarkeit und Funktionalität im Adobe Acrobat Reader überprüft werden. Daraus soll am Ende der Arbeit eine Übersicht der tatsächlichen Möglichkeiten eines Angreifers mit JavaScript in PDF-Dateien erstellt werden.

Die vorher bekannten Sicherheitsmechanismen aus der Dokumentation werden zuerst erläutert, da sie ausschlaggebend für den anschließenden Testaufbau und die entwickelte Methodik zum Testen der API-Aufrufe sind. Danach werden die Ergebnisse kategorienweise ausgewertet, woraus zum Schluss eine Gesamtübersicht entsteht.

### 4.1 Sicherheitsmechanismen

Adobe definiert zu Beginn ihrer JavaScript API Reference zwei Mechanismen, die die Sicherheit der API-Aufrufe gewährleisten sollen. Diese werden in den beiden folgenden Abschnitten erläutert. Außerdem bietet Adobe Acrobat die Möglichkeit, PDF-Dateien mit einem Zertifikat zu signieren und somit Vertrauen zwischen zwei Parteien zu generieren.

#### 4.1.1 Privilegierter Kontext

Sicherheitsbeschränkte API-Aufrufe werden in der Dokumentation mit einem roten „S“ in der kleinen Tabelle gekennzeichnet. Sie können aus Sicherheitsgründen nur in einem privilegierten Kontext ausgeführt werden, der Konsolen-, Batch- und Anwendungsinitialisierungsereignisse umfasst. Durch das Signieren des Dokuments mit einem vertrauenswürdigen Zertifikat kann ebenfalls ein privilegierter Kontext geschaffen werden. Alle anderen Events (siehe Abschnitt 2.4) gelten als nicht-privilegiert.

### 4.1.2 Secure Path

Für API-Aufrufe, die Daten auf eine lokale Festplatte des Systems schreiben, gilt das Konzept des „sicheren Pfades“. Damit kann auf keinen systemkritischen Ordner wie z. B. *root* oder *windows* zugegriffen werden.

## 4.2 Testumgebung

Für die Tests der API-Aufrufe wird geeignete Software benötigt. Dafür muss der Zustand dieser Software ermittelt und festgehalten werden. Außerdem müssen Zertifikate für die verschiedenen Testfälle erstellt werden, da einige API-Aufrufe nur in einem privilegierten Kontext ausgeführt werden können.

### 4.2.1 Software

Zur Erstellung der PDF-Dateien wurde Adobe Acrobat Pro 2017 mit der Version 17.011.30156 verwendet. Diese Software ist ein kommerzielles Produkt der Adobe Acrobat Familie und bietet vollen Umfang zur Erstellung und Bearbeitung von PDF-Dateien.

Zum Testen der PDF-Dateien wurde der Adobe Acrobat Reader DC mit der Version 19.021.20058 benutzt. Alle Einstellungen wurden auf ihren Standardwerten belassen.

### 4.2.2 Standardeinstellungen für JavaScript im Adobe Acrobat Reader DC

Unter dem Menüeintrag „Einstellungen“ des Reiters „Bearbeiten“ befinden sich im Adobe Acrobat Reader DC auch die Einstellungen für JavaScript (Abb. 4.1). Standardmäßig sind die Punkte *Acrobat JavaScript aktivieren* und *Sicherheitsrichtlinie für globale Objekte aktivieren* aktiviert. Die Punkte *Menübefehlen Berechtigung zur Ausführung von JavaScript erteilen* und *Konsole bei Fehlern und Meldungen anzeigen* sind deaktiviert.

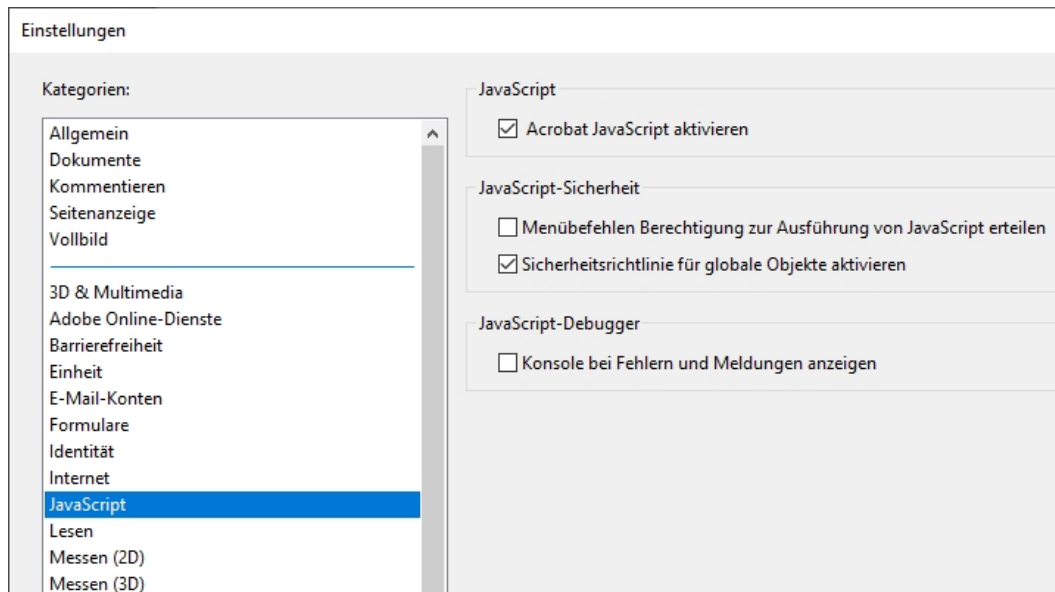


Abbildung 4.1: JavaScript Standardeinstellungen im Adobe Acrobat Reader DC

## 4.3 Durchführung

### 4.3.1 Erstellung der PDF-Dateien

Um die gleichen Voraussetzungen für alle Tests zu schaffen, wurde eine Testvorlage angefertigt. Eine leere PDF-Datei kann mit dem API-Aufruf `app.newDoc()` in der Konsole erstellt werden. Die Konsole kann durch die Tastenkombination **[Strg + J]** geöffnet werden. Die PDF-Datei wurde automatisch als PDF-Version 1.6 erzeugt. Für die Vorlage wurde in der leeren PDF-Datei ein Formular-Button erstellt, der den zu testenden API-Aufruf ausführt. Einige Aufrufe benötigen ein zusätzliches Element, auf dem der JavaScript-Code operiert, um dessen Funktionalität zu demonstrieren. Darauf wird in der Beschreibung der jeweiligen Tests näher eingegangen.

### 4.3.2 Erstellung der Zertifikate

PDF-Dateien können mit einem Zertifikat versehen werden, um Vertrauen zwischen zwei Parteien aufzubauen. Dazu werden sie mit einer digitalen ID signiert (digitale Signatur) [20]. Anschließend können API-Aufrufe in einem privilegierten Kontext ausgeführt werden, falls das Zertifikat es erlaubt. Ein neues Zertifikat kann mit einem Adobe Acrobat Produkt erstellt werden. Hier wurde der Adobe Acrobat Pro

2017 dafür benutzt. Dazu wird eine neue digitale ID in den Einstellungen für digitale IDs und vertrauenswürdige Zertifikate hinzugefügt. Die Einstellung befindet sich unter dem Punkt „Unterschriften“ in den Einstellungen des Acrobat Pro 2017. Mit der Erstellung einer digitalen ID wird automatisch ein dazugehöriges, selbst signiertes Zertifikat eingerichtet.

Die digitale ID wird im Standardformat PKCS#12 erstellt und beinhaltet Name, Abteilung, Firma, E-Mail-Adresse, Land und Schlüsselalgorithmus. Zuletzt muss noch ein Passwort festgelegt werden, das zum Unterschreiben und Entschlüsseln von Dateien benötigt wird.

Für die Erstellung der Zertifikate wurde bewusst ein anderes Acrobat Produkt als der Reader gewählt, um die beiden Parteien simulieren zu können.

### 4.3.3 Zertifikate hinzufügen

Damit die zertifizierten Dokumente bei den Tests vom Adobe Acrobat Reader als vertrauenswürdig anerkannt werden, müssen die Zertifikate dem Programm hinzugefügt werden. Dafür muss das Zertifikat der erstellten digitalen ID aus dem Adobe Acrobat Pro 2017 exportiert werden, um es anschließend in die Liste der vertrauenswürdigen Zertifikate im Adobe Acrobat Reader DC zu importieren. Die Liste befindet sich ebenfalls in den Einstellungen für digitale IDs und vertrauenswürdige Zertifikate. Beim Importierungsvorgang kann die Vertrauenswürdigkeit deklariert werden. Dabei kann für ein Zertifikat einer der drei folgenden Zwecke ausgewählt werden, für die das Zertifikat bei der Unterschriftvalidierung als vertrauenswürdig gilt (Abb. 4.2):

- Dynamischer Inhalt
- Eingebettete JavaScripts mit hoher Berechtigungsstufe
- Privilegierte Systemvorgänge (Netzwerk, Drucken, Dateizugriff usw.)

Damit die Signaturprüfung der Zertifikate erfolgreich ist, muss die Option *Dieses Zertifikat als vertrauenswürdigen Stamm verwenden* gesetzt werden.

Vor allem der Zweck *Eingebettete JavaScripts mit hoher Berechtigungsstufe* sollte dafür sorgen, dass ein API-Aufruf in einem privilegierten Kontext ausgeführt werden kann. Um alle Fälle abzudecken und somit die Auswirkungen der beiden anderen Einstellungsmöglichkeiten auf die API-Aufrufe zu untersuchen, wird für jeden der drei Zwecke eine digitale ID mit einem Zertifikat erstellt und der Anwendung hinzugefügt. Die Zertifikate tragen jeweils den Namen des Zwecks, damit sie nachher einfacher zu unterscheiden sind.

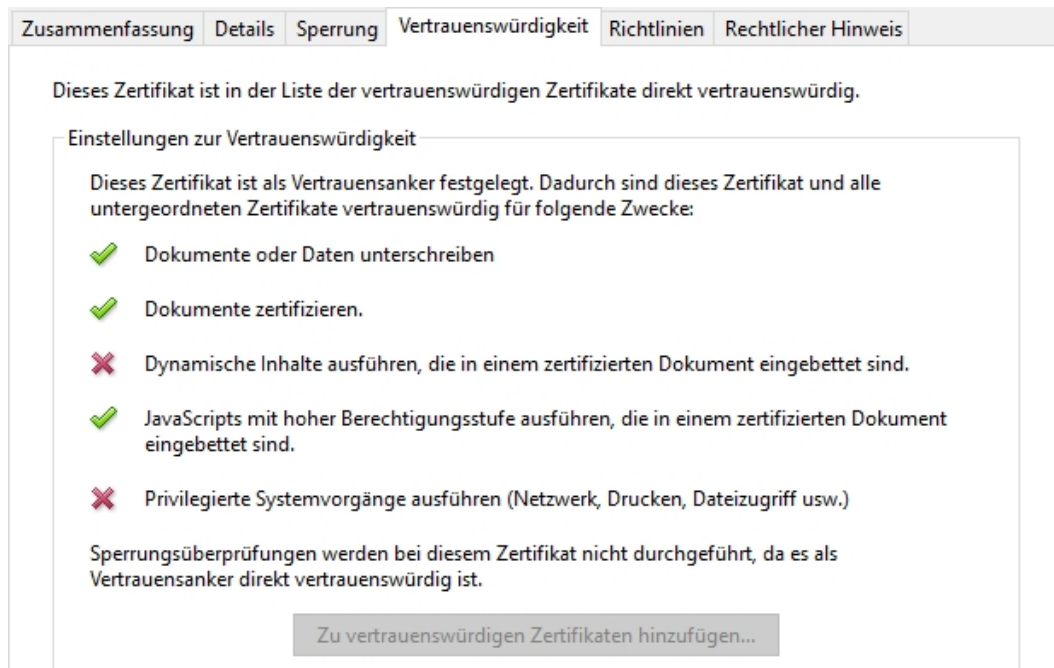


Abbildung 4.2: Vertrauenswürdigkeit eines Zertifikates

#### 4.3.4 Testfälle

Um festzustellen, welche der sicherheitskritischen API-Aufrufe in dem Angriffszenario (Abschnitt 3.3) ausgeführt werden und welche dabei in einem privilegierten Kontext ausgeführt werden müssen, wurden insgesamt vier Testfälle für jeden API-Aufruf erstellt. Diese umfassen PDF-Dateien, die mit einem der drei verschiedenen Zertifikate signiert wurden und eine nicht zertifizierte PDF-Datei:

- Testfall 1: PDF-Datei ohne Zertifikat
- Testfall 2: PDF-Datei mit Zertifikat, das dynamische Inhalte erlaubt
- Testfall 3: PDF-Datei mit Zertifikat, das eingebettete JavaScripts mit hoher Berechtigungsstufe erlaubt
- Testfall 4: PDF-Datei mit Zertifikat, das privilegierte Systemvorgänge erlaubt

#### 4.3.5 Testablauf

Für jeden API-Aufruf wurde ein einfacher JavaScript Code geschrieben, der im Idealfall nur den jeweiligen API-Aufruf enthält. Für einige API-Aufrufe wurde ein weiteres Element benötigt, um sie korrekt auszuführen. Diese wurden den jeweiligen Testdokumenten hinzugefügt und werden in den Testergebnissen erwähnt. Außerdem

dienen sie zur Demonstration der Aktion eines API-Aufrufs. Zur weiteren Veranschaulichung wurde bei geeigneten API-Aufrufen die Konsole geöffnet, um die bewirkten Änderungen anzuzeigen. Die Konsole lässt sich unter normalen Umständen eigentlich nicht im Adobe Acrobat Reader öffnen, da sie hauptsächlich für Debugging in Acrobat und Acrobat Pro gedacht ist. Um die Konsole dennoch aufzurufen, gibt es zwei Möglichkeiten: Es wird eine PDF-Datei mit bewusst fehlerhaftem JavaScript Code erstellt und diese im Adobe Acrobat Reader geöffnet. Dazu muss jedoch die JavaScript Einstellung *Konsole bei Fehlern und Meldungen anzeigen* vorher aktiviert werden. Die zweite Möglichkeit besteht darin, eine PDF-Datei zu erstellen, die den Code `console.show()` ausführt. Für die Testdateien wurde einfachheitshalber letzteres in Kombination mit `console.println()` verwendet.

Die fertigen JavaScript Codes wurden dem Button in der Vorlage als „Mouse Up“ Event (siehe Abschnitt 2.4) hinzugefügt. Da alle Events, bis auf Konsolen-, Batch- und Anwendungsinitialisierungsevents, über die gleichen Berechtigungen verfügen, ist es theoretisch egal, wo der JavaScript Code platziert wird. Besonderheiten, die für API-Aufrufe in diesem Bereich gelten, wurden in Kapitel 3.5 beschrieben.

Anschließend wurden die vorbereiteten Dateien mit dem Adobe Acrobat Reader DC geöffnet und der zu testende API-Aufruf ausgeführt. Warnmeldungen und Besonderheiten bei der Ausführung wurden zu jedem Aufruf in einer Dokumentation der Ergebnisse festgehalten.

## 5 Testergebnisse

Im Folgenden werden die Testergebnisse dargestellt und die ausführbaren API-Aufrufe der Kategorien präsentiert. Eine vollständige Dokumentation der Ergebnisse befindet sich im Anhang A.

### 5.1 Sicherheitswarnungen

Fast alle API-Aufrufe, die nicht ausgeführt werden konnten, gaben eine Sicherheitswarnung aus.

```
NotAllowedError: Sicherheitseinstellungen verhindern den Zugriff auf diese Eigenschaft oder
Methode.
Doc.insertPages:5:Field Button:Mouse Up
```

Listing 5.1: NotAllowedError Sicherheitswarnung

Die Meldung macht es deutlich, dass der API-Aufruf aus Sicherheitsgründen nicht ausgeführt werden konnte. Die letzte Zeile gibt den entsprechenden API-Aufruf und die Quelle in Form von Objekt und Event an. Diese Sicherheitswarnung wird jedoch nur in der Konsole ausgegeben und ist damit für den Benutzer nicht sichtbar, solange er die JavaScript Einstellung *Konsole bei Fehlern und Meldungen anzeigen* nicht aktiviert hat. Da diese Einstellung standardmäßig nicht aktiviert ist, wird bei der folgenden Auswertung angenommen, dass der Anwender es nicht mitbekommt, wenn eine PDF-Datei versucht, einen sicherheitskritischen API-Aufruf auszuführen. In der Statistik fallen diese API-Aufrufe unter die Kategorie API-Aufruf wurde nicht ausgeführt und es wurde keine Sicherheitswarnung ausgegeben. Alle anderen API-Aufrufe, die eine sichtbare Sicherheitswarnung erzeugen, werden in der Statistik zusammengefasst. Auf die einzelnen verschiedenen Sicherheitswarnungen wird in der folgenden Präsentation der Ergebnisse gesondert eingegangen.

### 5.2 Auswertung

Von den 121 sicherheitskritischen API-Aufrufen konnten insgesamt 82 ausgeführt werden. Davon erzeugten 39 eine Sicherheitswarnung, die dem Benutzer angezeigt wurde.

### 5.2.1 Testfall 1: PDF-Datei ohne Zertifikat

Insgesamt konnten 61 API-Aufrufe in einem Dokument ohne Zertifikat ausgeführt werden. 38 davon erzeugten eine sichtbare Sicherheitswarnung.

|                               |                              |
|-------------------------------|------------------------------|
| app launchURL                 | Doc getURL                   |
| Doc submitForm                | Doc print                    |
| PrintParams interactive       | Doc exportDataObject         |
| app addToolButton             | app alert                    |
| app execDialog                | app popUpMenuEx              |
| app response                  | Doc closeDoc                 |
| app execMenuItem              | app setInterval              |
| app setTimeout                | app formsVersion             |
| app platform                  | app plugIns                  |
| app printerNames              | app viewerType               |
| app viewerVariation           | app viewerVersion            |
| app.media version             | app.media getPlayers         |
| Doc path                      | app listMenuItems            |
| ocg setAction                 | Doc addField                 |
| Doc addAnnot                  | Field currentValueIndices    |
| Field defaultValue            | Field display                |
| Field password                | Field value                  |
| Field checkThisBox            | Field deleteItemAt           |
| Field insertItemAt            | Field setItems               |
| Field exportValue             | Field defaultIsChecked       |
| Field setAction               | Annotation author            |
| Annotation contents           | Annotation hidden            |
| Annotation lock               | Annotation modDate           |
| Annotation readOnly           | Annotation destroy           |
| Annotation setProps           | Annotation transitionToState |
| Collab addStateModel          | Annotation state             |
| Annotation inReplyTo          | Annotation type              |
| Doc delay                     | Annotation delay             |
| Field delay                   | Bookmark execute             |
| Collab documentToStream       | Doc getDataObjectContents    |
| annotAttachment contentStream |                              |

Tabelle 5.2: API-Aufrufe, die ohne Zertifikat ausgeführt werden können

Wie in Tabelle 5.2 zu sehen ist, konnte der API-Aufruf `this.exportDataObject()` erfolgreich ausgeführt werden. Um die Methode zu testen, wurde zur Vorbereitung die Textdatei `test.txt` an die PDF-Datei angehängt. Bei den anschließenden Tests wurde vor der Ausführung der angehängten Datei bei allen Testfällen eine Sicher-

heitswarnung angezeigt. Diese warnte den Nutzer vor möglichen Risiken beim Öffnen einer unbekannten Datei (siehe Abb. 5.3). Die Warnung konnte mit der Option „das Öffnen von Dateien dieses Typs immer zulassen“ zusätzlich ausgewählt/bestätigt werden. Eine weitere Auswahlmöglichkeit war „das Öffnen von Dateien dieses Typs niemals zulassen“, wonach der Vorgang abgebrochen wurde.

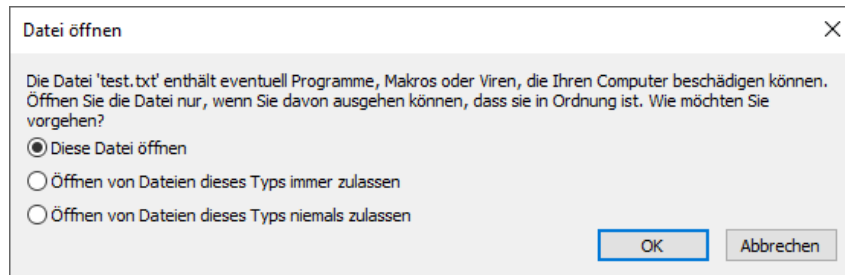


Abbildung 5.3: Sicherheitswarnung: Datei öffnen

Ohne Zertifikat konnten zwar keine Menüeinträge verändert werden, aber es war möglich, einen **ToolButton** zu erstellen. Dieser erscheint rechts in der Werkzeugleiste als Eintrag und in der linken oberen Ecke als Button. Beide werden explizit als JavaScript-Fenster gekennzeichnet, außer in Testfall 3. Das gleiche Verhalten ist bei den Dialogfenstern, die mit dem Benutzer interagieren, zu sehen (siehe Abb. 5.4).

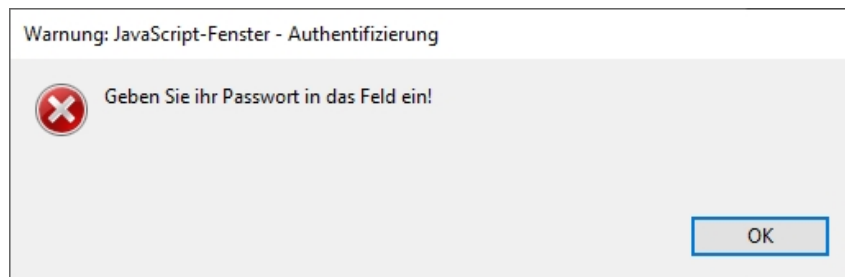


Abbildung 5.4: Sicherheitswarnung: JavaScript-Fenster

Die Methoden `this.closeDoc()`, `this.execMenuItem()`, `app.setInterval()` und `app.setTimeout()` wurden alle ohne eine Sicherheitswarnung in allen Testfällen ausgeführt. `this.closeDoc()` konnte ein Dokument erfolgreich schließen, ohne die Änderung zu speichern.

In Testfall 3 konnten bei der Methode `app.execMenuItem()` noch zusätzlich Menüeinträge ausgeführt werden, die nicht in der Liste der erlaubten Einträge vermerkt waren. So ließ sich zum Beispiel der Menüeintrag **Open** damit aktivieren.

Durch eine normale PDF-Datei ließen sich insgesamt 11 API-Aufrufe ausführen, die nützliche Informationen über die Anwendung oder das System offenlegten. Darunter

befand sich der Aufruf `app.printerNames`, der eine Liste an verfügbarer Drucker Soft- und Hardware ausgab. So konnte Name und Typ eines am System angeschlossenen, physischen Druckers identifiziert werden, wie auch installierte Software wie das Programm *PDF24 Creator*<sup>1</sup>, da es als virtueller Drucker über einen Gerätetreiber im Betriebssystem installiert war.

Weitere Informationen über Software auf dem System lieferte die Methode `app.media.getPlayers()`, indem die verfügbaren MediaPlayer aufgelistet wurden. Darunter befanden sich auch Informationen über die Version und die unterstützten Wiedergabeformate der jeweiligen Player.

Informationen über die aktuell verwendete Anwendung konnten mit Hilfe der Aufrufe `app.viewerType`, `app.viewerVariation` und `app.viewerVersion` erhalten werden. `app.pluginIns` gab genaue Informationen über die installierten PlugIns des Readers, wie Installationspfad und Versionsnummer preis.

Mit einem nicht zertifizierten Dokument ließen sich auch viele Änderungen an einem Dokument vornehmen. Die meisten davon bezogen sich auf Annotationen und Formularfelder, jedoch erschien beim Schließen des Dokuments immer eine Sicherheitswarnung (siehe Abb. 5.5), die den Benutzer fragt, ob er die Änderungen speichern möchte. So könnten dem Dokument neue Formularfelder mit `this.addField()` hinzugefügt und bereits existierende Felder mit den anderen Aufrufen manipuliert werden.

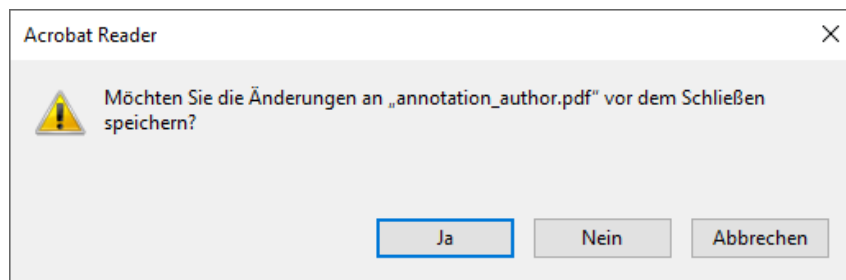


Abbildung 5.5: Sicherheitswarnung: Speicherabfrage

Der vom Benutzer eingetragene Wert eines Textfeldes ließ sich mit `f.value` auf einen beliebigen Wert ändern und der aktuell ausgewählte Wert einer Listbox ließ sich mit `f.currentValueIndices` auf einen anderen Wert aus der Liste umstellen. Mit Hilfe der Methoden `f.setItems()`, `f.insertItemAt()` und `f.deleteItemAt()` konnten die Einträge einer Listbox oder Combobox beliebig geändert werden, indem sie gelöscht oder neu hinzugefügt wurden.

Auch Checkboxes konnten nachträglich verändert werden, indem durch `f.checkThisBox()` der aktuelle Zustand und mit `f.defaultIsChecked()` der Standardzustand auf aktiviert oder deaktiviert gesetzt wurde. Außerdem konnten die `exportValues` verändert werden.

---

<sup>1</sup><https://de.pdf24.org/>

Bei einem ausfüllbaren Textfeld kann die Eigenschaft `f.password` auf `true` gesetzt werden, um den eingetragenen Wert mit Asterix Symbolen zu maskieren und ihn beim Schließen des Dokuments nicht zu speichern. In einem Test gelang es, diese Eigenschaft nachträglich auf `false` zu ändern, womit der eingetragene Wert doch noch gespeichert wurde.

Für Annotationen konnten vergleichbare Daten verändert werden. So war es möglich, Autor, Inhalt und Status nachträglich zu ändern. Durch den Aufruf von `a.modDate` konnte sogar das Datum der letzten Bearbeitung geändert werden.

Mit `Collab.addState()` ließ sich ein neues Statusmodell erstellen und der Status einer Annotation konnte auf einen Wert des neuen Modells gesetzt werden. Allerdings ist das Statusmodell nicht persistent und verschwindet bei einem Neustart der Anwendung wieder. Bei erneutem API-Aufruf wird der zuvor gesetzte Zustand aus dem neuen Statusmodell wieder angezeigt.

Der Wert der Eigenschaft `a.state` konnte zwar ohne Sicherheitswarnung überschrieben werden, wurde aber nicht als Status der Annotation angezeigt. Um den Status erfolgreich zu ändern, konnte die Methode `a.transitionToState()` ausgeführt werden.

Alle Manipulationen an einer Annotation waren nur in einem nicht zertifizierten Dokument möglich, da die Einstellung der Zertifikate eine nachträgliche Änderung an Kommentaren verbietet. Dies ist die Standardeinstellung bei Zertifikaten.

### 5.2.2 Testfall 2: Zertifikat erlaubt dynamische Inhalte

Testfall 1 und Testfall 2 erzeugten die selben Ergebnisse, außer bei `this.addField()` und den API-Aufrufen für die Manipulation von Annotationen. Ersteres wird bei Testfall 1 ausgeführt, bei Testfall 2 aber nicht. Letzteres ist auf die Einstellung des Zertifikats zurückzuführen. Das lässt darauf schließen, dass das Erlauben dynamischer Inhalte in einem Dokument keine großen Auswirkungen auf die Ausführung von JavaScript-Inhalten hat.

### 5.2.3 Testfall 3: Zertifikat erlaubt JavaScript-Skripte mit hoher Berechtigungsstufe

Von den ausführbaren Aufrufen benötigten 21 ein zertifiziertes PDF-Dokument, um ausgeführt zu werden. 18 API-Aufrufe davon ließen sich nur dann anwenden, wenn das Dokument mit einem Zertifikat, das eingebetteten JavaScript-Code mit hoher Berechtigungsstufe erlaubt, signiert war (Tab. 5.6).

|                             |                                    |
|-----------------------------|------------------------------------|
| Net.Discovery queryServices | Net.Discovery resolveService       |
| this.SaveAs                 | global setPersistent               |
| app.addMenuItem             | app.addSubMenu                     |
| app.hideMenuItem            | security.validateSignaturesOnOpen  |
| app.getPath                 | SecurityHandler setPasswordTimeout |
| DirConnection search        | identity.corporation               |
| identity.email              | identity.loginName                 |
| identity.name               | SecurityHandler loginPath          |
| SecurityHandler digitalIDs  | this.disclosed                     |

Tabelle 5.6: API-Aufrufe, die nur mit dem Zertifikat aus Testfall 3 (JavaScript-Code mit hoher Berechtigungsstufe) ausgeführt werden konnten

Die Net.Discovery API-Aufrufe waren die Einzigen aus der Kategorie Netzwerkdienste (siehe Abschnitt 3.5.1.2), die überhaupt ausgeführt werden konnten. Im Rahmen dieser Arbeit konnte aus Aufwandsgründen jedoch keine genauere Aussage über die Ausgabe getroffen werden.

Der Test der Methode `this.SaveAs()` ergab, dass ein Dokument erfolgreich auf dem System gespeichert werden konnte. Vor dem Speichervorgang wurde jedoch eine Sicherheitswarnung (siehe Abb. 5.7) angezeigt, die dem Nutzer mitteilt, dass eine Schreibberechtigung erforderlich ist. In dem Dialogfenster der Warnung war der vorher festgelegte Speicherpfad zu sehen. Der Vorgang konnte dann bestätigt oder abgelehnt werden. Das gespeicherte Dokument enthielt auch das gültige Zertifikat. Bei dem Versuch, die Datei beim Speichern in ein docx Dokument zu konvertieren, zeigte die Konsole den Fehler an, dass eine nicht implementierte oder veraltete Funktion aufgerufen wurde.

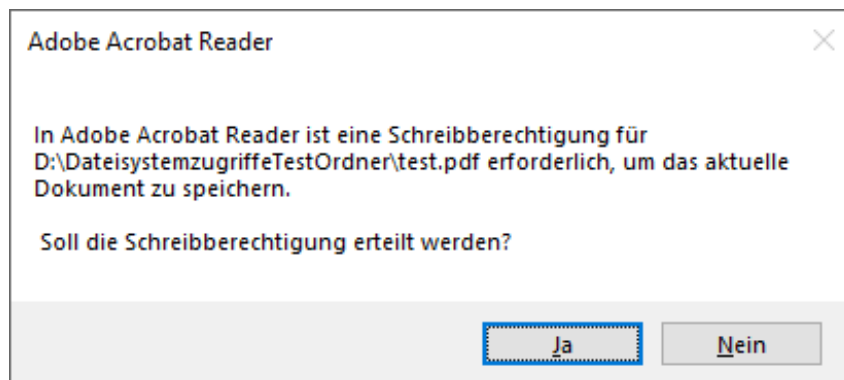


Abbildung 5.7: Sicherheitswarnung: Schreibberechtigung

Durch das Erlauben von JavaScript-Code mit hoher Berechtigungsstufe konnte auch das User-Interface manipuliert werden. Mit `app.hideMenuItem()` konnte ein bereits

existierender Menüeintrag entfernt und anschließend mit `app.addItem()` ein neuer hinzugefügt werden. Der eingetragene JavaScript Code wurde bei der Betätigung des Menüpunkts problemlos ausgeführt.

Die Offenlegung einiger Informationen bedurfte ebenfalls eines Zertifikats dieser Art. So konnte der Pfad des Ordners, der bei der Installation erstellt wurde, nur unter diesen Umständen mit der Methode `app.getPath()` ausgelesen werden.

In diesem Kontext konnte auch die Methode `search()` aufgerufen werden, welche eine Liste mit den verfügbaren Nutzer-Entitäten ausgab. Diese Methode kann nur auf einem security handler ausgeführt werden, welcher zuvor mit `security.getHandler()` ausgewählt wurde. Das ist ebenfalls nur in einem privilegierten Kontext möglich. Für den Test wurde der security handler **adobe.AAB** ausgewählt, der die lokal gespeicherten Entitäten enthält.

Über diesen security handler konnten damit auch die **digitalen IDs** und der **login Pfad** mit den korrespondierenden Methoden aus der Tabelle 5.6 ausgelesen werden. Falls ein Anwender im Adobe Acrobat Reader mit seinem Account angemeldet ist, können die Informationen aus dem **identity** Objekt offengelegt werden.

Die einzige Dokumenteigenschaft, die nur in diesem Testfall geändert werden konnte, ist die **disclosed** Eigenschaft. Damit kann ein Dokument für JavaScript-Skripte in anderen Dokumenten zugänglich gemacht werden.

#### 5.2.4 Testfall 4: Zertifikat erlaubt privilegierte Systemvorgänge

Zwei weitere API-Aufrufe der 21, die nur in einem zertifizierten Dokument ausgeführt werden können, benötigen ein Zertifikat, das privilegierte Systemvorgänge zulässt. Das betrifft die Methoden `app.media.getURL()` und `app.media.getURLSettings()`.

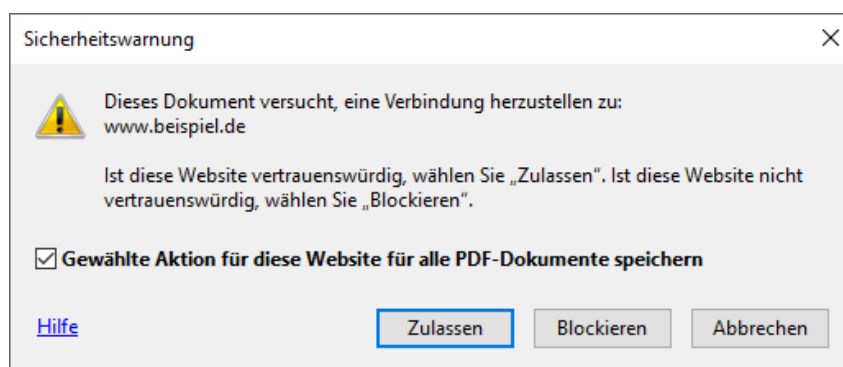


Abbildung 5.8: Sicherheitswarnung: URL-Aufruf

Diese konnten nur in diesem Kontext ungehindert ausgeführt werden. In den anderen Testfällen wurde der Anwender zunächst darüber informiert, dass das Dokument

versucht, eine Verbindung aufzubauen (siehe Abb. 5.8). Der Benutzer hat dann die Möglichkeit, die Aktion zuzulassen, zu blockieren oder abubrechen. Wenn der Anwender die Aktion ohne Weiteres zulässt, wird die gewählte Aktion für diese Webseite für alle PDF-Dokumente standardmäßig gespeichert. Diese Option kann jedoch auch deaktiviert werden. In diesem Fall führte die Bestätigung jedoch vorerst zu einem Abbruch des Vorgangs und es wurde eine weitere Sicherheitswarnung (siehe Abb. 5.9) ausgegeben. Anschließend hatte der Benutzer aber die Option, dem Dokument einmalig oder immer zu vertrauen, wonach die Aktion schließlich doch ausgeführt wurde. Diese Option wird über dem Dokument in einem gelben Balken angezeigt, die dem Nutzer mitteilt, dass einige Funktionen deaktiviert wurden, um mögliche Sicherheitsprobleme zu vermeiden. Die Aktion sollte nur dann aktiviert werden, wenn dem Dokument vertraut wird.

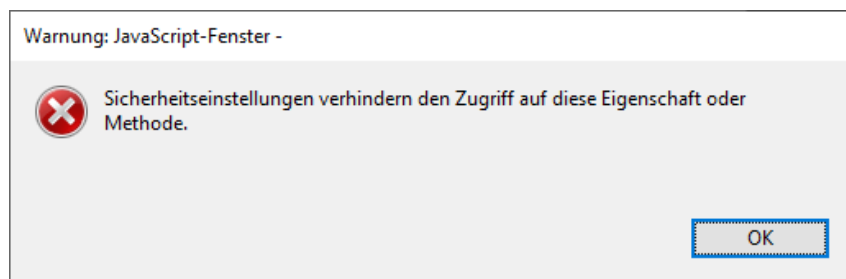


Abbildung 5.9: Sicherheitswarnung: Zugriff verhindert

### 5.2.5 Gemeinsamkeiten von Testfall 3 und 4

Der letzte API-Aufruf konnte in den Testfällen 3 und 4 ohne Sicherheitswarnung aufgerufen werden. Mit `app.activeDocs` war es möglich, ein Array zu erhalten, des-

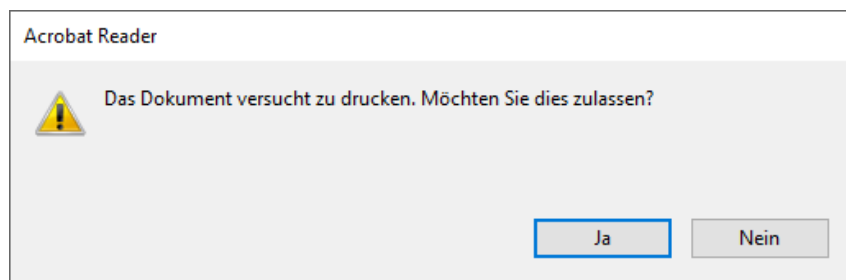


Abbildung 5.10: Sicherheitswarnung: Druckvorgang

sen Einträge den aktuell geöffneten Dokumenten in der Anwendung entsprechen. Die Einträge daraus konnten genutzt werden, um auf die anderen Dokumente zuzugreifen. Damit können die API-Aufrufe des `Doc`-Objekts auf diese Dokumente

angewendet werden. Zum Beispiel war es damit möglich, die **disclosed** Eigenschaft dieser Dateien auf **true** zu setzen.

Eine weitere Gemeinsamkeit dieser beiden Fälle war bei der Ausführung der API-Aufrufe `this.print()`, `PrintParams interactive`, `app.launchURL()` und `this.getURL()` zu sehen. Im Gegensatz zu den anderen beiden Fällen wurde hier keine Sicherheitswarnung ausgegeben und die Aktion wurde sofort ausgeführt (siehe Abb. 5.10 und Abb. 5.8).

### 5.2.6 Nicht ausführbare API-Aufrufe

Insgesamt konnten 39 API-Aufrufe gar nicht ausgeführt werden (siehe Abb. 5.11).

|                             |                                  |
|-----------------------------|----------------------------------|
| app loadPolicyFile          | Net.SOAP connect                 |
| Net.SOAP request            | Net.SOAP response                |
| Net.HTTP request            | security exportToFile            |
| security importFromFile     | app mailMsg                      |
| Doc mailDoc                 | Doc mailForm                     |
| app newDoc                  | Doc baseURL                      |
| Doc info                    | Doc addScript                    |
| Doc certifyInvisibleSign    | Doc setAction                    |
| Doc setPageAction           | Doc deletePages                  |
| Template spawn              | Doc createDataObject             |
| Doc setDataObjectContents   | Doc addReceipientListCryptFilter |
| Doc applyRedactions         | Link setAction                   |
| Doc addLink                 | Doc addWatermarkFromText         |
| ocg initState               | ocg locked                       |
| Doc insertPages             | Doc replacePages                 |
| Doc addWatermarkFromFile    | Doc importDataObject             |
| Field signatureSetSeedValue | Field SignatureSign              |
| Annotation cAttachmentPath  | Bookmark createChild             |
| Bookmark setAction          | Field setLock                    |
| app trustedFunction         |                                  |

Tabelle 5.11: API-Aufrufe, die nicht ausgeführt werden konnten

Das betrifft vor allem viele Aufrufe aus den Unterkategorien *Dokumenteigenschaftenmanipulation* (siehe Abschnitt 3.21) und *Dokumentinhaltmanipulation* (siehe Abschnitt 3.5.4.2), die eine Aktion für das Dokument setzen oder den Inhalt eines Datenobjekts ändern. Außerdem wurden die meisten Methoden mit einem Dateisystemzugriff nicht ausgeführt. Die Methode für die Rechteauserweiterung ließ sich ebenfalls nicht ausführen. Die Aufrufe `app.loadPolicyFile()` und `f.setLock()` zeigten

zudem kein Reaktion. Es wurde auch kein Sicherheitswarnung oder sonstige Fehlermeldung ausgegeben.

## 6 Zusammenfassung

Der Test der sicherheitskritischen API-Aufrufe zeigt, dass 50%, nämlich 61 von 121, der sicherheitskritischen API-Aufrufe ohne ein Zertifikat ausgeführt werden konnten. Insgesamt zeigten davon 38 eine für den Benutzer sichtbare Sicherheitswarnung an. Dabei variiert die Auswirkung der Meldungen sehr stark.

Bei der Sicherheitswarnung für URL-Aufrufe wird der Benutzer zwar gewarnt, dass das Dokument versucht, eine Verbindung herzustellen und gibt dabei auch die aufzurufende URL an, aber die Option, die gewählte Aktion für diese Webseite für alle PDF-Dokumente zu speichern, ist standardmäßig aktiviert. Das trägt nicht unbedingt zur Sicherheit bei, da diese Meldungen oftmals auch aus Versehen bestätigt werden, weswegen die Option zunächst deaktiviert sein sollte.

Eine ähnliche Sicherheitswarnung wurde bei der Methode `this.exportDataObject()`, die eine angehängte Datei ausführt, angezeigt. Hier wird die Aktion deutlich besser behandelt, da zunächst auch auf mögliche Risiken hingewiesen wird, aber der Standardwert steht auf „Diese Datei öffnen“ und nicht auf „Öffnen von Dateien dieses Typs immer zulassen“, was eine weitere Auswahlmöglichkeit darstellt.

Ein positiver Aspekt ist die Sicherheitswarnung bei Toolbuttons und Dialogboxen, welche mit dem Hinweis **Warnung: JavaScript-Fenster** versehen wird, sofern das erzeugende Dokument keine eingebetteten JavaScripts mit hoher Berechtigungsstufe (Testfall 3) erlaubt. Zugleich zeigt dieser Fall auch eine negative Seite, da dies die einzigen Fälle sind, wo diese Meldung auftaucht. Der Benutzer sollte an viel mehr Stellen darauf aufmerksam gemacht werden, dass JavaScript ausgeführt wird.

Die häufigste Sicherheitsmeldung tritt beim Schließen eines Dokuments auf, wenn darin zuvor etwas verändert wurde. Bei den Tests betraf das hauptsächlich Änderungen an Formularfeldern und Annotationen. Der Anwender kann dann darüber entscheiden, die Änderungen zu speichern, zu verwerfen oder den Vorgang abzubrechen. Vor allem bei PDF-Dateien mit Formularfeldern ist es sehr wahrscheinlich, dass ein Benutzer Eingaben getätigt hat und beim Schließen des Dokuments so eine Abfrage erhält. Dabei fällt es dann nicht auf, dass vor dem Schließen noch etwas per JavaScript geändert wurde. Hier sollte auch eine Warnung ausgegeben werden, dass eine JavaScript Aktion ausgeführt werden soll. Auch ohne eine vorherige Veränderung durch den Benutzer am Dokument ist diese Speicherabfrage eher unauffällig.

Fast alle API-Aufrufe, die nicht ausgeführt wurden, erzeugten keine für den Benutzer sichtbare Sicherheitswarnung. Der Benutzer sollte darauf hingewiesen werden, falls ein Dokument versucht, unerlaubte JavaScript-Aktionen auszuführen. Dann könnte die Quelle des Dokuments überprüft werden, um weiteren Dokumenten zu misstrauen und sie gegebenenfalls zu sperren. So bleiben die Versuche unerkannt und es kann darauf nicht reagiert werden.

Dateisystemzugriffe waren bis auf `this.saveAs()` nicht möglich und wurden bei einem vorher angegebenen Pfad entweder nicht ausgeführt oder öffneten einen Dateibrowser, was einen unbemerkten Zugriff verhindert.

Dafür konnte an einigen Stellen durch Aufrufe wie `this.setAction()` nachträglich beliebiger JavaScript-Code hinzugefügt werden, was die Sicherheit definitiv beeinträchtigt. Darunter fällt auch die Möglichkeit, eine JavaScript-Aktion zu einem beliebigen Zeitpunkt einmalig oder in einem gewählten Intervall ausführen zu lassen.

Die Offenlegung einiger Informationen, wie zum Beispiel die Liste der verfügbaren Drucker, ist fragwürdig und sollte auf ihren Nutzen und Relevanz geprüft werden. Das Gleiche gilt für das temporäre Abspeichern von Dokumentinhalten.

Am gefährlichsten sind die Aufrufe, die Formularfelder und Annotation verändern, da sie beim Schließen eines Dokuments angewendet werden können und somit nachträglich den Inhalt und vor Allem Benutzereingaben verändern. Damit könnten verfälschte Informationen weitergegeben werden, die eventuell bei wichtigen und offiziellen Dokumenten größere Auswirkungen mit sich ziehen. Die nachträglichen Änderungen an Annotationen lassen sich durch Zertifikateinstellungen verbieten und wurden im Zuge der Tests deswegen auch nicht ausgeführt. Die Zertifikateinstellung kann beim Signieren einer Datei auf **keine Änderungen** gesetzt werden, womit keine Änderungen an Formularfeldern mehr möglich sind. Jedoch können Formularfelder damit nicht mehr ausgefüllt werden, was dem Sinn und Zweck in den meisten Situationen entgegensteht.

Ein weiterer sicherheitskritischer API-Aufruf ist der `app.activeDocs` Aufruf, da durch ihn auf andere in der Anwendung geöffnete Dokumente zugegriffen werden kann. Jedoch lässt er sich nur in den Testfällen 3 und 4 ausführen, was durch das Zertifikat immerhin einen gewissen Grad an Vertrauen benötigt.

Die Zertifikate sind eine gute Basis, um Vertrauen zwischen zwei Parteien aufzubauen und sind ein allgemein akzeptiertes Mittel, um Authentizität und Integrität für ein Dokument zu gewährleisten. So kann das Sicherheitsprinzip des privilegierten Kontextes realisiert werden, indem das Zertifikat aus Testfall 3 benutzt wird. Jedoch sollten mehr sicherheitskritische API-Aufrufe nur durch ein zertifiziertes Dokument ausgeführt werden können. Das würde zwar die allgemein geschätzte Vielseitigkeit

eines PDF-Dokuments etwas einschränken, dafür aber die Sicherheit für den Benutzer erhöhen.

Ein gutes Mittel zur Erhöhung der Sicherheit war im Fall der Methoden `app.media.getURLData()` und `app.media.getURLSettings()` zu sehen. Diese wurden zunächst nicht ausgeführt, wonach eine Warnung erschien, die dem Nutzer mitteilt, dass einige Funktionen deaktiviert wurden, um mögliche Sicherheitsprobleme zu vermeiden. Die Funktionen sollen nur dann aktiviert werden, wenn dem Dokument vertraut wird. Danach hat der Benutzer die Option, dem Dokument einmalig oder immer zu vertrauen. Eine solche Warnung sollte dem Benutzer immer angezeigt werden, wenn ein Dokument JavaScript beinhaltet. Dann kann sich ein Benutzer das Dokument zunächst anschauen und nach eigenem Ermessen entscheiden, ob er die JavaScript-Funktionen aktivieren möchte.



## 7 Fazit und Ausblick

Die Ergebnisse dieser Arbeit zeigen, dass sicherheitskritische JavaScript-Aktionen auf dem System des Benutzers ausgeführt werden können. Dadurch lassen sich Dokumentinhalte nachträglich ändern, URL-Aufrufe und angehängte Dateien ausführen, Informationen über die Anwendung und das System offenlegen und beliebige JavaScript-Skripte einfügen. Außerdem kann auf andere geöffnete Dateien in der Anwendung zugegriffen werden.

Bisherige Arbeiten auf diesem Gebiet haben sich häufig auf die Umgehung von Berechtigungen und der Ausführung von beliebigem, schadhaftem JavaScript-Code konzentriert. Diese Arbeit zeigt, welche anwendungsspezifischen Aktionen mit Hilfe von JavaScript auf dem System eines Opfers ausgeführt werden können und in welchem Kontext dies möglich ist. Sicherheitswarnungen wurden nicht in allen Fällen ausgegeben und informierten den Benutzer nicht ausreichend. Dadurch weiß ein Angreifer nun, welche Angriffe möglicherweise nicht auffallen.

Durch den Einsatz von selbst signierten Zertifikaten konnte ermittelt werden, welche API-Aufrufe eine bestimmte Berechtigung brauchen, um ausgeführt zu werden. In diesem Bereich könnte weitere Forschung betrieben werden, indem Zertifikate von verschiedenen Zertifizierungsstellen betrachtet werden. Des Weiteren wäre die Auswirkung in Zertifikatsketten interessant. Außerdem könnte überprüft werden, bei welchen Änderungen eine Signatur nicht mehr valide ist.

Die API-Aufrufe wurden jeweils einzeln getestet. In weiteren Tests könnten verschiedene Kombinationen ausprobiert werden. Das Testen aller implementierten Events hätte den Rahmen dieser Arbeit überschritten.

Diese Arbeit betrachtet explizit die Ausführung von JavaScript im Adobe Acrobat Reader DC. Die sicherheitskritischen API-Aufrufe könnten in anderer Reader-Software, die JavaScript unterstützen, getestet werden, beispielsweise Foxit Reader. Da werden möglicherweise andere JavaScript-APIs verwendet. Des Weiteren könnten die API-Aufrufe im Adobe PDF-Browser-Plug-in für verschiedene Webbrowser untersucht werden.



# Abbildungsverzeichnis

|      |                                                                 |    |
|------|-----------------------------------------------------------------|----|
| 3.15 | Toolbutton mit Warnung: JavaScript-Fenster . . . . .            | 21 |
| 4.1  | JavaScript Standardeinstellungen im Adobe Acrobat Reader DC . . | 35 |
| 4.2  | Vertrauenswürdigkeit eines Zertifikates . . . . .               | 37 |
| 5.3  | Sicherheitswarnung: Datei öffnen . . . . .                      | 41 |
| 5.4  | Sicherheitswarnung: JavaScript-Fenster . . . . .                | 41 |
| 5.5  | Sicherheitswarnung: Speicherabfrage . . . . .                   | 42 |
| 5.7  | Sicherheitswarnung: Schreibberechtigung . . . . .               | 44 |
| 5.8  | Sicherheitswarnung: URL-Aufruf . . . . .                        | 45 |
| 5.9  | Sicherheitswarnung: Zugriff verhindert . . . . .                | 46 |
| 5.10 | Sicherheitswarnung: Druckvorgang . . . . .                      | 46 |

# Tabellenverzeichnis

|      |                                                                                                                                           |    |
|------|-------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.5  | API-Aufrufe der Kategorie: URI-Aufrufe . . . . .                                                                                          | 16 |
| 3.7  | API-Aufrufe der Kategorie: Netzwerkdienste . . . . .                                                                                      | 17 |
| 3.8  | API-Aufrufe der Kategorie: Dateisystemzugriffe . . . . .                                                                                  | 18 |
| 3.10 | API-Aufrufe der Kategorie: Programmausführung . . . . .                                                                                   | 19 |
| 3.11 | API-Aufrufe der Kategorie: Ausführung einer beliebigen Datei mit<br>Dateisystemzugriff . . . . .                                          | 20 |
| 3.13 | API-Aufrufe der Kategorie: UI-Manipulation . . . . .                                                                                      | 20 |
| 3.16 | API-Aufrufe der Kategorie: Anwendungseigenschaftenmanipulation .                                                                          | 21 |
| 3.17 | API-Aufrufe der Kategorie: UI-Interaktion . . . . .                                                                                       | 22 |
| 3.18 | API-Aufrufe der Kategorie: Ausführung einer Anwendungsaktion . .                                                                          | 23 |
| 3.19 | API-Aufrufe der Kategorie: Automatisierte Codeausführung . . . . .                                                                        | 23 |
| 3.20 | API-Aufrufe der Kategorie: Offenlegung von Informationen . . . . .                                                                        | 23 |
| 3.21 | API-Aufrufe der Kategorie: Dokumenteigenschaftenmanipulation . .                                                                          | 25 |
| 3.23 | API-Aufrufe der Kategorie: Dokumentinhaltmanipulation . . . . .                                                                           | 26 |
| 3.24 | API-Aufrufe der Kategorie: Dokumentinhaltmanipulation mit Datei-<br>systemzugriff . . . . .                                               | 27 |
| 3.25 | API-Aufrufe der Kategorie: Feldmanipulation . . . . .                                                                                     | 28 |
| 3.26 | API-Aufrufe der Kategorie: Annotationmanipulation . . . . .                                                                               | 29 |
| 3.28 | API-Aufrufe der Kategorie: Annotationmanipulation mit Dateisys-<br>temzugriff . . . . .                                                   | 30 |
| 3.29 | API-Aufrufe der Kategorie: Verzögerte Ausführungen . . . . .                                                                              | 30 |
| 3.30 | API-Aufrufe der Kategorie: Bookmarks . . . . .                                                                                            | 31 |
| 3.31 | API-Aufrufe der Kategorie: Zugriff auf andere Dokumente . . . . .                                                                         | 31 |
| 3.32 | API-Aufrufe der Kategorie: Temporäres Abspeichern eines Dokumentin-<br>halts . . . . .                                                    | 32 |
| 5.2  | API-Aufrufe, die ohne Zertifikat ausgeführt werden können . . . . .                                                                       | 40 |
| 5.6  | API-Aufrufe, die nur mit dem Zertifikat aus Testfall 3 (JavaScript-<br>Code mit hoher Berechtigungsstufe) ausgeführt werden konnten . . . | 44 |
| 5.11 | API-Aufrufe, die nicht ausgeführt werden konnten . . . . .                                                                                | 47 |

## Liste der Listings

|      |                                                                                                  |    |
|------|--------------------------------------------------------------------------------------------------|----|
| 2.1  | Syntax eines JavaScript API-Aufrufs . . . . .                                                    | 8  |
| 2.2  | Beispiel eines Catalog-Objekts einer PDF-Datei . . . . .                                         | 8  |
| 2.3  | API-Aufruf für das Ereignis Page/Open . . . . .                                                  | 10 |
| 3.1  | API-Aufruf des App Objekts, der ein leeres Dokuments erstellt. . . .                             | 14 |
| 3.2  | API-Aufruf des Doc Objekts, der im Dokument ein Formularfeld erzeugt. . . . .                    | 14 |
| 3.3  | API-Aufruf des Field Objekts, der das Feld mit dem Namen „fTest“ ausblendet. . . . .             | 14 |
| 3.4  | API-Aufruf des Annotation Objekts, der den Inhalt einer Annotation ändert. . . . .               | 14 |
| 3.6  | URL-Aufruf mit der Methode <code>app.launchURL</code> . . . . .                                  | 16 |
| 3.9  | Die Variable <code>test</code> wird auf den Wert „1“ gesetzt und persistent gespeichert. . . . . | 19 |
| 3.12 | Die Methode speichert die Datei „test.txt“ temporär und öffnet sie. .                            | 20 |
| 3.14 | Die Methode <code>app.addItem()</code> fügt einen neuen Menüeintrag hinzu. .                     | 21 |
| 3.22 | Beispiel zur Methode <code>this.setAction</code> . . . . .                                       | 25 |
| 3.27 | Beispiel zu den API-Aufrufen <code>a.modDate</code> und <code>a.contents</code> . . . . .        | 30 |
| 5.1  | NotAllowedError Sicherheitswarnung . . . . .                                                     | 39 |

# Literaturverzeichnis

- [1] Common vulnerabilities and exposures. <https://cve.mitre.org/>.
- [2] Daniel Bachfeld. PDF-Dokumente als Angriffsvektor. 09 2006. <https://www.heise.de/security/meldung/PDF-Dokumente-als-Angriffsvektor-163747.htm>.
- [3] Aniello Castiglione, Alfredo De Santis, and Claudio Soriente. Security and privacy issues in the portable document format. *Journal of Systems and Software*, 83:1813–1822, 10 2010. doi: 10.1016/j.jss.2010.04.062.
- [4] Stuart Cheshire and Marc Krochmal. DNS-Based Service Discovery. RFC 6763, February 2013. URL <https://rfc-editor.org/rfc/rfc6763.txt>.
- [5] Roberto Chinnici, Jean-Jacques Moreau, Arthur Ryman, and Sanjiva Weerawarana. Web services description language (wsdl) version 2.0 part 1: Core language. World Wide Web Consortium, Recommendation REC-wsdl20-20070626, June 2007.
- [6] Igino Corona, Davide Maiorca, Davide Ariu, and Giorgio Giacinto. Lux0r: Detection of malicious pdf-embedded javascript code through discriminant analysis of api references. volume 2014, 11 2014. doi: 10.1145/2666652.2666657.
- [7] ECMA International. *Standard ECMA-262 - ECMAScript Language Specification*. 5.1 edition, June 2011. URL <http://www.ecma-international.org/publications/standards/Ecma-262.htm>.
- [8] Valentin Hamon. Malicious uri resolving in pdf documents. *J. Comput. Virol.*, 9(2):65–76, May 2013. ISSN 1772-9890. doi: 10.1007/s11416-013-0179-2. URL <https://doi.org/10.1007/s11416-013-0179-2>.
- [9] Abdul-Aziz Hariri and Edgar Pek. Tackling Privilege Escalation With Offense And Defense, 2019. <https://i.blackhat.com/eu-19/Thursday/eu-19-Hariri-Tackling-Privilege-Escalation-With-Offense-And-Defense.pdf>.
- [10] Adobe Inc. *Acrobat DC SDK Documentation*, . [https://help.adobe.com/en\\_US/acrobat/acrobat\\_dc\\_sdk/2015/HTMLHelp/index.html#rhsyns=%20&t=Acro12\\_MasterBook%2FIntroduction\\_Help\\_TitlePage%2FAbout\\_This\\_Help.htm](https://help.adobe.com/en_US/acrobat/acrobat_dc_sdk/2015/HTMLHelp/index.html#rhsyns=%20&t=Acro12_MasterBook%2FIntroduction_Help_TitlePage%2FAbout_This_Help.htm).

- [11] Adobe Inc. Security (digital signatures), . <https://www.adobe.com/devnet-docs/acrobatetk/tools/PrefRef/Windows/Security.html>.
- [12] Adobe Systems Incorporated. Document management — portable document format — part 1: Pdf 1.7. ISO 32000-1:2008, July 2008.
- [13] Alexander Inführ. Multiple pdf vulnerabilities - text and pictures on steroids, Dezember 2014. <https://insert-script.blogspot.com/2014/12/multiple-pdf-vulnerabilites-text-and.html>.
- [14] Anish Karmarkar, Yves Lafon, Henrik Frystyk Nielsen, Marc Hadley, Noah Mendelsohn, Jean-Jacques Moreau, and Martin Gudgin. SOAP version 1.2 part 1: Messaging framework (second edition). W3C recommendation, W3C, April 2007. <http://www.w3.org/TR/2007/REC-soap12-part1-20070427/>.
- [15] Pavel Laskov and Nedin Srndic. Static detection of malicious javascript-bearing pdf documents. pages 373–382, 12 2011. doi: 10.1145/2076732.2076785.
- [16] Daiping Liu, Haining Wang, and Angelos Stavrou. Detecting malicious javascript in pdf through document instrumentation. pages 100–111, 06 2014. doi: 10.1109/DSN.2014.92.
- [17] Davide Maiorca and Battista Biggio. Digital investigation of PDF files: Unveiling traces of embedded malware. *CoRR*, abs/1707.05102, 2017. URL <http://arxiv.org/abs/1707.05102>.
- [18] Witold Pryjda. Windows: Pdf & java machen 66% aller angriffe aus. 04 2013. <https://winfuture.de/news,79162.html>.
- [19] Frédéric Raynal, Guillaume Delugré, and Damien Aumaitre. Malicious origami in pdf. *Journal in Computer Virology*, 6(4):289–315, Nov 2010. ISSN 1772-9904. doi: 10.1007/s11416-009-0128-2. URL <https://doi.org/10.1007/s11416-009-0128-2>.
- [20] Jörg Schwenk. *Sicherheit und Kryptographie im Internet: Theorie und Praxis*. Lehrbuch. Springer Vieweg, Wiesbaden, 4., überarb. und erw. Aufl. edition, 2014. ISBN 9783658065430.
- [21] Charles Smutz and Angelos Stavrou. Malicious pdf detection using metadata and structural features. pages 239–248, 12 2012. doi: 10.1145/2420950.2420987.
- [22] PDF Association staff. Pdf in 2016: Broader, deeper, richer. 12 2015. <https://www.pdfa.org/pdf-in-2016-broader-deeper-richer/>.



# A Anhang: Testergebnisse

Die folgenden Tabellen zeigen die vollständigen Ergebnisse aller API-Aufrufe. Sie befinden sich in der gleichen Reihenfolge, wie in Abschnitt 3.5.

Für die Zeilen gelten folgende Konventionen:

**API-Aufruf:** Der untersuchte API-Aufruf im Format: Objektname Methode/Eigenschaft Name des API-Aufrufs.

**Beschreibung:** Eine kurze Beschreibung der Aktion des API-Aufrufs.

**Reader Tests:** Ergebnisse der einzelnen Testfälle.

**normal:** Testfall 1, PDF-Datei ohne Zertifikat.

**dynamisch:** Testfall 2, PDF-Datei mit Zertifikat, das dynamische Inhalte erlaubt.

**eingebettet:** Testfall 3, PDF-Datei mit Zertifikat, das eingebettete JavaScripts mit hoher Berechtigungsstufe erlaubt.

**privilegiert:** Testfall 4, PDF-Datei mit Zertifikat, das privilegierte Systemvorgänge erlaubt.

**„Ja“:** API-Aufruf wurde ausgeführt. Es gibt keine für den Benutzer sichtbare Sicherheitswarnung.

**„Nein“:** API-Aufruf wurde nicht ausgeführt. Es gibt keine für den Benutzer sichtbare Sicherheitswarnung.

**„SW“:** Es wurde eine für den Benutzer sichtbare Sicherheitswarnung ausgegeben.

**„keine Reaktion“:** Die Ausführung des API-Aufrufs ergab keine sichtbare Reaktion.

**Rote Zellen:** Der API-Aufruf in dieser Zeile kann laut der *JavaScript für Acrobat API-Referenz* nur in einem privilegierten Kontext ausgeführt werden.

**Graue Zellen:** Der API-Aufruf in dieser Zeile kann laut der *JavaScript für Acrobat API-Referenz* im Adobe Acrobat Reader nicht ausgeführt werden.

## URI-Aufrufe

| API-Aufruf                       | Beschreibung                                                                                                                                                   | Reader Tests                                                                                                                               |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| app Methode launchURL            | Öffnet eine URL in einem Browserfenster.                                                                                                                       | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja<br>privilegiert: Ja                                                                 |
| app.media Methode getUrlData     | Liefert ein MediaData-Objekt , das Daten repräsentiert, die für eine URL und einen optionalen MIME-Typ abgerufen werden sollen.                                | normal: Nein, SW<br>dynamisch: Nein, SW<br>eingebettet: Nein, SW<br>privilegiert: Ja                                                       |
| app.media Methode getUrlSettings | Nimmt ein PlayerArgs-Objekt, das eine Einstellungseigenschaft enthält und gibt ein MediaSettings-Objekt zurück, das für die Wiedergabe einer URL geeignet ist. | normal: Nein, SW<br>dynamisch: Nein, SW<br>eingebettet: Nein, SW<br>privilegiert: Ja                                                       |
| Doc Methode getUrl               | Ruft die angegebene URL über das Internet mit einem GET auf.                                                                                                   | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja<br>privilegiert: Ja                                                                 |
| Doc Methode submitForm           | Schickt das Formular zu einer spezifizierten URL.                                                                                                              | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW                                                         |
| app Methode loadPolicyFile       | Läd eine cross-domain policy von einem per URL angegebenen Speicherort.                                                                                        | normal: Nein, keine Reaktion<br>dynamisch: Nein, keine Reaktion<br>eingebettet: Nein, keine Reaktion<br>privilegiert: Nein, keine Reaktion |

## Netzwerkdienste

| API-Aufruf                           | Beschreibung                                                                                                                              | Reader Tests                                                               |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Net.SOAP Methode connect             | Konvertiert die URL eines WSDL-Dokuments in ein JavaScript-Objekt mit aufrufbaren Methoden entsprechend dem Webservice.                   | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Net.SOAP Methode request             | Leitet einen Remote Procedure Call (RPC) ein oder sendet eine XML-Nachricht an einen SOAP-HTTP-Endpunkt.                                  | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Net.SOAP Methode response            | Leitet einen Remote Procedure Call (RPC) ein oder sendet eine XML-Nachricht an einen SOAP-HTTP-Endpunkt, ohne auf eine Antwort zu warten. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Net.Discovery Methode queryServices  | Lokalisiert Netzwerkdienste, die selbst mit DNS Service Discovery (DNS-SD) veröffentlicht wurden.                                         | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein   |
| Net.Discovery Methode resolveService | Ermöglicht die Bindung eines Dienstnamens an eine Netzwerkadresse und einen Port, damit eine Verbindung hergestellt werden kann.          | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein   |
| Net.HTTP Methode request             | Ruft asynchron einen HTTP Web Service auf.                                                                                                | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |

# Dateisystemzugriffe

| API-Aufruf                      | Beschreibung                                                                                                                                                                                                                                                                                                   | Reader Tests                                                                 |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Doc Methode saveAs              | Speichert die Datei in dem geräteunabhängigen Pfad, der durch den erforderlichen Parameter cPath angegeben ist.                                                                                                                                                                                                | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja, SW<br>privilegiert: Nein |
| global Methode setPersistent    | Steuert, ob eine angegebene Variable über Aufrufe von Acrobat hinweg persistent ist. Beim Beenden der Anwendung werden persistente globale Variablen in der Datei glob.js gespeichert, die sich im Ordner des Benutzers für Skripte auf Ordner Ebene befindet und beim Start der Anwendung neu geladen werden. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein     |
| security Methode exportToFile   | Exportiert ein Zertifikat Objekt auf eine lokale Festplatte als eine rohe Zertifikatsdatei.                                                                                                                                                                                                                    | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein   |
| security Methode importFromFile | Liest eine Rohdaten-Datei und gibt die Daten als Objekt mit einem durch cType angegebenen Typ zurück. Die zu importierende Datei muss ein gültiges Zertifikat sein.                                                                                                                                            | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein   |

# Programmausführung

| API-Aufruf                          | Beschreibung                                                                                                                                                                               | Reader Tests                                                               |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| app Methode mailMsg                 | Sendet eine E-Mail-Nachricht mit oder ohne Benutzerinteraktion. Für bUI=false ohne Benutzerinteraktion.                                                                                    | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode mailDoc                 | Speichert das aktuelle PDF-Dokument und versendet es als Anhang an alle Empfänger, mit oder ohne Benutzerinteraktion. Für bUI=false ohne Benutzerinteraktion.                              | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode mailForm                | Exportiert die Formulardaten und versendet die resultierende FDF-Datei als Anhang an alle Empfänger, mit oder ohne Benutzerinteraktion. bUI = false um ohne Benutzerinteraktion zu senden. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode print                   | Druckt alle oder die angegebenen Seiten des Dokuments. bUI= false verhindert, dass dem User ein UI angezeigt wird, bSilent=true verhindert das Anzeigen des Cancel Dialogs.                | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja<br>privilegiert: Ja |
| PrintParams Eigenschaft interactive | Gibt den Grad der Interaktion zwischen dem Benutzer und dem Druckauftrag an.                                                                                                               | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja<br>privilegiert: Ja |

## Ausführung einer beliebigen Datei mit Dateisystemzugriff

| API-Aufruf                      | Beschreibung                                                                                              | Reader Tests                                                                       |
|---------------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Doc Methode<br>exportDataObject | Extrahiert das angegebene Datenobjekt in eine temporäre Datei und öffnet es wenn nLaunch = 2 gesetzt ist. | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |

## UI-Manipulation

| API-Aufruf                | Beschreibung                                                                                                     | Reader Tests                                                                   |
|---------------------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| app Methode addMenuItem   | Fügt einem Menü einen Menüeintrag hinzu.                                                                         | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein       |
| app Methode addSubMenu    | Fügt der Anwendung ein Menü mit einem Untermenü hinzu.                                                           | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein       |
| app Methode addToolButton | In Acrobat wird dem Bereich Plugin Addon Tools auf der rechten Seite des Anzeigebereichs ein Button hinzugefügt. | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja<br>privilegiert: Ja, SW |
| app Methode hideMenuItem  | Entfernt einen Menüeintrag.                                                                                      | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein       |

## Anwendungseigenschaftenmanipulation

| API-Aufruf                                       | Beschreibung                                                                                                                     | Reader Tests                                                             |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| security Eigenschaft<br>validateSignaturesOnOpen | Liest oder setzt die Benutzereinstellung, die bewirkt, dass Signaturen beim Öffnen eines Dokuments automatisch überprüft werden. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein |
| SecurityHandler Methode<br>setPasswordTimeout    | Legt die Anzahl der Sekunden fest, nach denen das Passwort zwischen den Signaturen ablaufen soll.                                | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein |

## UI-Interaktion

| API-Aufruf              | Beschreibung                                                                                                        | Reader Tests                                                                   |
|-------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| app Methode alert       | Zeigt eine alert Dialogbox an.                                                                                      | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja<br>privilegiert: Ja, SW |
| app Methode execDialog  | Stellt dem Benutzer ein modales Dialogfeld zur Verfügung.                                                           | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja<br>privilegiert: Ja, SW |
| app Methode popUpMenuEx | Erstellt ein Popup-Menü an der aktuellen Mausposition.                                                              | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja             |
| app Methode response    | Zeigt ein Dialogfenster mit einer Frage und einem Eingabefeld für die Beantwortung der Frage durch den Benutzer an. | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja<br>privilegiert: Ja, SW |

## Ausführung einer Anwendungsaktion

| API-Aufruf               | Beschreibung                                                                                           | Reader Tests                                                               |
|--------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| app Methode newDoc       | Erstellt ein neues Dokument und gibt sein Doc-Objekt zurück.                                           | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode closeDoc     | Schließt ein Dokument. Mit bNoSave=true wird das Dokument ohne Speichern und ohne Warnung geschlossen. | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja         |
| app Methode execMenuItem | Führt den ausgewählten Menüpunkt aus.                                                                  | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja         |

## Automatisierte Codeausführung

| API-Aufruf              | Beschreibung                                                                                                              | Reader Tests                                                       |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| app Methode setInterval | Gibt ein JavaScript-Skript und einen Zeitraum an. Das Skript wird jedes Mal ausgeführt, wenn der Zeitraum abgelaufen ist. | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja |
| app Methode setTimeout  | Gibt ein JavaScript-Skript und einen Zeitraum an. Das Skript wird nach Ablauf des Zeitraums nur einmal ausgeführt.        | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja |

## Offenlegung von Informationen

| API-Aufruf                                                                                     | Beschreibung                                                                                                               | Reader Tests                                                             |
|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| app Eigenschaft formsVersion                                                                   | Gibt die Versionsnummer der Forms Software des Viewers an.                                                                 | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| app Eigenschaft platform                                                                       | Die Plattform, auf der das Skript gerade ausgeführt wird.                                                                  | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| app Eigenschaft plugIns<br>plugIns Eigenschaften:<br>certified, loaded, name, path,<br>version | Ein Array von PlugIn-Objekten, die die Plugins darstellen, die derzeit im Viewer installiert sind.                         | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| app Eigenschaft printerNames                                                                   | Eine Liste der verfügbaren Drucker.                                                                                        | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| app Eigenschaft viewerType                                                                     | Gibt an, welche Vieweranwendung ausgeführt wird.                                                                           | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| app Eigenschaft viewerVariation                                                                | Gibt an, welche Version der Anwendung genutzt wird(Bsp. Business Tool).                                                    | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| app Eigenschaft viewerVersion                                                                  | Gibt an, welche Viewer Version benutzt wird.                                                                               | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| app.media Eigenschaft version                                                                  | Versionsnummer der multimedia API.                                                                                         | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| app.media Methode getPlayer                                                                    | Liefert ein PlayerInfoList-Objekt, das ein Array von PlayerInfo-Objekten ist, die die verfügbaren Media-Player darstellen. | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| Doc Eigenschaft path                                                                           | Geräteunabhängiger Pfad des Dokuments.                                                                                     | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| app Methode getPath                                                                            | Gibt die Pfade zu den Ordnern aus, die bei der Installation erstellt wurden.                                               | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein |

## Offenlegung von Informationen

|                                        |                                                                                                                     |                                                                          |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| app Methode listMenuItems              | Gibt ein Array von TreeItem Objekten zurück, die die Menü Hierarchie beschreiben.                                   | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja       |
| DirConnection Methode search           | Durchsucht das Verzeichnis und gibt ein Array mit User-Entität-Objekten zurück, die den Suchparametern entsprechen. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein |
| identity Eigenschaft corporation       | Der Unternehmensname, den der Benutzer im Bereich Identitätseinstellungen eingegeben hat.                           | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein |
| identity Eigenschaft email             | Die E-Mail-Adresse, die der Benutzer im Bereich Identitätseinstellungen eingegeben hat.                             | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein |
| identity Eigenschaft loginName         | Der Anmeldenname, der vom Betriebssystem registriert wurde.                                                         | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein |
| identity Eigenschaft name              | Der Benutzername, den der Benutzer im Bereich Identitätseinstellungen eingegeben hat.                               | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein |
| SecurityHandler Eigenschaft loginPath  | Der geräteunabhängige Pfad zur Profildatei des Benutzers, mit dem er sich am Sicherheitshandler angemeldet hat.     | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein |
| SecurityHandler Eigenschaft digitalIDs | Die Zertifikate, die den aktuell ausgewählten digitalen IDs für diesen Security-Handler zugeordnet sind.            | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein |

# Dokumenteigenschaftenmanipulation

| API-Aufruf                       | Beschreibung                                                                                                                                                 | Reader Tests                                                               |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Doc Eigenschaft baseUrl          | Die Basis-URL für das Dokument wird verwendet, um relative Weblinks innerhalb des Dokuments aufzulösen.                                                      | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Eigenschaft disclosed        | Gibt an, ob das Dokument für JavaScript-Skripte in anderen Dokumenten zugänglich sein soll.                                                                  | normal: Nein<br>dynamisch: Nein<br>eingebettet: Ja<br>privilegiert: Nein   |
| Doc Eigenschaft info             | Gibt ein Objekt mit Eigenschaften aus dem Dokumentinformationswörterbuch in der PDF-Datei an. Beim Adobe Reader können keine Eigenschaften verändert werden. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode addScript            | Setzt ein Dokument Level Skript. Das Skript wird beim Öffnen des Dokuments ausgeführt.                                                                       | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode certifyInvisibleSign | Fügt dem Dokument ein unsichtbares Zertifikat hinzu.                                                                                                         | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode setAction            | Setzt die JavaScript-Aktion des Dokuments für einen bestimmten Trigger.                                                                                      | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode setPageAction        | Setzt die JavaScript-Aktion der Seite für einen bestimmten Trigger.                                                                                          | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |

# Dokumentinhaltmanipulation

| API-Aufruf                                             | Beschreibung                                                                                                                                | Reader Tests                                                               |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Doc Methode deletePages                                | Löscht Seiten eines Dokuments.                                                                                                              | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Template Methode spawn /<br>Doc Methode createTemplate | Erzeugt eine template basierte neue Seite im Dokument. / Erstellt ein Template der ausgewählten Seite(n).                                   | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode<br>createDataObject                        | Erstellt ein Datenobjekt.                                                                                                                   | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode<br>setDataObjectContents                   | Ersetzt den durch den Parameter cName angegebenen Dateianhang durch den Inhalt des oStream-Parameters.                                      | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode<br>addRecipientListCryptFilter             | Fügt dem Dokument einen Kryptofilter hinzu. Der Kryptofilter wird zur Verschlüsselung von Datenobjekten verwendet.                          | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode<br>applyRedactions                         | Wendet die Redaktionsmarkierungen auf das Dokument an und entfernt den unterliegenden Inhalt. Entfernt optional die Markierung auch selbst. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Link Methode setAction                                 | Setzt die angegebene JavaScript-Aktion für den MouseUp-Trigger für das Link-Objekt.                                                         | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| ocg Eigenschaft initState                              | Mit dieser Eigenschaft wird festgelegt, ob dieses OCG standardmäßig ein- oder ausgeschaltet ist.                                            | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| ocg Methode setAction                                  | Speichert einen JavaScript-Ausdruck, der nach jeder Zustandsänderung für dieses OCG ausgewertet werden soll.                                | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja         |
| Doc Methode addLink                                    | Fügt einen neuen Link hinzu.                                                                                                                | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |

## Dokumentinhaltmanipulation

|                                  |                                                                                                                                                                                  |                                                                              |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Doc Methode addField             | Erstellt ein neues Formular Feld.                                                                                                                                                | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode addAnnot             | Erstellt eine Annotation mit den spezifizierten Eigenschaften.                                                                                                                   | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode addWatermarkFromText | Fügt einen Text als Wasserzeichen zu den angegebenen Seiten im Dokument hinzu und platziert das Wasserzeichen in einer optionalen Inhaltsgruppe (ocg).                           | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein   |
| ocg Eigenschaft locked           | Anhand dieser Eigenschaft wird bestimmt, ob dieses OCG gesperrt ist. Wenn ein OCG gesperrt ist, kann sein Ein/Aus-Zustand nicht über die Benutzeroberfläche umgeschaltet werden. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein   |

## Dokumentinhaltmanipulation mit Dateisystemzugriff

| API-Aufruf                       | Beschreibung                                                                                                                                           | Reader Tests                                                               |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Doc Methode insertPages          | Fügt ausgewählte Seiten des Quelldokuments in das aktuelle Dokument.                                                                                   | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode replacePages         | Ersetzt Seiten im aktuellen Dokument durch Seiten aus dem Quelldokument.                                                                               | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode addWatermarkFromFile | Fügt eine Seite als Wasserzeichen zu den angegebenen Seiten im Dokument hinzu und platziert das Wasserzeichen in einer optionalen Inhaltsgruppe (ocg). | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Doc Methode importDataObject     | Importiert eine externe Datei in das Dokument.                                                                                                         | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |

## Feldmanipulation

| API-Aufruf                            | Beschreibung                                                                                                                                                                                                                                                                              | Reader Tests                                                                       |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Field Eigenschaft currentValueIndices | Liest und schreibt einzelne oder mehrere Werte einer Listbox oder Combobox.                                                                                                                                                                                                               | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |
| Field Eigenschaft defaultValue        | Der Standardwert eines Feldes, d.h. der Wert, auf den das Feld gesetzt wird, wenn das Formular zurückgesetzt wird.                                                                                                                                                                        | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |
| Field Eigenschaft display             | Steuert, ob das Feld auf dem Bildschirm und im Druck ausgeblendet oder sichtbar ist.                                                                                                                                                                                                      | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |
| Field Eigenschaft password            | Gibt an, ob das Feld bei der Eingabe von Daten in das Feld Sternchen anzeigen soll. (Beim Senden werden die tatsächlich eingegebenen Daten gesendet.) Wenn diese Eigenschaft wahr ist, werden die Daten im Feld nicht gespeichert, wenn das Dokument auf der Festplatte gespeichert wird. | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |
| Field Eigenschaft value               | Der Wert des Feldes, den der Benutzer eingegeben hat.                                                                                                                                                                                                                                     | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |
| Field Methode checkThisBox            | Aktiviert oder deaktiviert das angegebene Widget.                                                                                                                                                                                                                                         | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |
| Field Methode deleteItemAt            | Löscht einen Eintrag in einer ComboBox oder ListBox.                                                                                                                                                                                                                                      | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |
| Field Methode insertItemAt            | Fügt einer ListBox oder ComboBox einen neuen Eintrag hinzu.                                                                                                                                                                                                                               | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |
| Field Methode setItems                | Legt die Liste der Elemente für eine ComboBox oder eine ListBox fest.                                                                                                                                                                                                                     | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |

## Feldmanipulation

|                                     |                                                                                                                                                                               |                                                                                                                                            |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Field Eigenschaft exportValues      | Ein Array von Zeichenketten die den Exportwert des Feldes repräsentieren.                                                                                                     | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW                                                         |
| Field Methode defaultIsChecked      | Setzt das angegebene Widget, das standardmäßig aktiviert oder deaktiviert werden soll.                                                                                        | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW                                                         |
| Field Methode setAction             | Setzt die JavaScript-Aktion des Feldes für einen bestimmten Trigger.                                                                                                          | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW                                                         |
| Field Methode setLock               | Bestimmt, welche Felder gesperrt werden sollen, wenn eine Signatur auf dieses Signaturfeld angewendet wird. An gesperrten Feldern können keine Änderungen vorgenommen werden. | normal: Nein, keine Reaktion<br>dynamisch: Nein, keine Reaktion<br>eingebettet: Nein, keine Reaktion<br>privilegiert: Nein, keine Reaktion |
| Field Methode signatureSetSeedValue | Legt Eigenschaften fest, die beim Signieren von Signaturfeldern verwendet werden.                                                                                             | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein                                                                 |
| Field Methode signatureSign         | Signiert ein Feld mit dem spezifizierten security handler.                                                                                                                    | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein                                                                 |

## Annotationmanipulation

| API-Aufruf                            | Beschreibung                                                                                                                                                                     | Reader Tests                                                                 |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Annotation Eigenschaft author         | Liest oder setzt den Autor der Annotation.                                                                                                                                       | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Annotation Eigenschaft contents       | Zugriff auf den Inhalt jeder Annotation, die ein Popup-Fenster hat. Für Audio- und Dateianhang-Annotationen wird der Text angegeben, der als Beschreibung angezeigt werden soll. | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Annotation Eigenschaft hidden, noView | Wenn true, wird die Annotation nicht angezeigt und es gibt keine Benutzerinteraktion, Anzeige oder Druck der Annotation.                                                         | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Annotation Eigenschaft lock           | Wenn true, ist die Annotation gesperrt, was dem Wert von readOnly entspricht                                                                                                     | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Annotation Eigenschaft modDate        | Das letzte Modifizierungsdatum der Annotation.                                                                                                                                   | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Annotation Eigenschaft readOnly       | Wenn true, sollte die Annotation angezeigt werden, aber nicht mit dem Benutzer interagieren.                                                                                     | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Annotation Methode destroy            | Zerstört die Annotation und entfernt sie von der Seite. Das Objekt wird ungültig.                                                                                                | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Annotation Methode setProps           | Setzt mehrere Eigenschaften für eine Annotation auf einmal.                                                                                                                      | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Annotation Methode transitionToState  | Setzt den Zustand der Annotation auf cState, indem ein Zustandswechsel durchgeführt wird.                                                                                        | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Annotation Eigenschaft state          | Der Status der Text Annotation.                                                                                                                                                  | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |

## Annotationmanipulation

|                                     |                                                                             |                                                                              |
|-------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Annotation Eigenschaft<br>inReplyTo | Gibt den Namen der<br>Annotation an, auf die diese<br>Annotation antwortet. | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Collab Methode<br>addStateModel     | Füg ein neues Zustandsmodell<br>zu Acrobat hinzu.                           | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja           |
| Annotation Eigenschaft type         | Ändert den Typ einer<br>Annotation oder ließt ihn aus.                      | normal: Ja, SW<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |

## Annotationmanipulation mit Dateisystemzugriff

| API-Aufruf                                | Beschreibung                                                                                                             | Reader Tests                                                               |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Annotation Eigenschaft<br>cAttachmentPath | Gibt den geräteunabhängigen<br>Pfad der anzuhängenden<br>Datei an. Falls nicht gesetzt,<br>öffnet sich der Dateibrowser. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |

## Verzögerte Ausführungen

| API-Aufruf                   | Beschreibung                                                                                                                                                      | Reader Tests                                                                       |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Doc Eigenschaft delay        | Verzögert die vorgenommenen Änderungen an Feldern.                                                                                                                | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja                 |
| Annotation Eigenschaft delay | Wenn true, werden Eigenschaftsänderungen an der Annotation in die Warteschlange gestellt und dann ausgeführt, wenn die Verzögerung wieder auf false gesetzt wird. | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |
| Field Eigenschaft delay      | Verzögert die Aktualisierung des Erscheinungsbildes eines Feldes.                                                                                                 | normal: Ja, SW<br>dynamisch: Ja, SW<br>eingebettet: Ja, SW<br>privilegiert: Ja, SW |

## Bookmarks

| API-Aufruf                      | Beschreibung                                         | Reader Tests                                                               |
|---------------------------------|------------------------------------------------------|----------------------------------------------------------------------------|
| Bookmark Methode<br>createChild | Erstellt ein neues<br>untergeordnetes Lesezeichen.   | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |
| Bookmark Methode execute        | Führt die Aktion eines<br>Lesezeichens aus.          | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja         |
| Bookmark Methode<br>setAction   | Setzt eine JavaScript Aktion<br>für ein Lesezeichen. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |

## Zugriff auf andere Dokumente

| API-Aufruf                               | Beschreibung                                                                                                          | Reader Tests                                                                                           |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>app</b> Eigenschaft <b>activeDocs</b> | Ein Array, dass das Doc-Objekt für jedes aktive Dokument enthält. Damit kann auf andere Dokumente zugegriffen werden. | normal: Nein, keine Reaktion<br>dynamisch: Nein, keine Reaktion<br>eingebettet: Ja<br>privilegiert: Ja |

## Temporäres Abspeichern eines Dokuments

| API-Aufruf                                   | Beschreibung                                                                                                                                                                                      | Reader Tests                                                       |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Collab Methode<br>documentToStream           | Speichert eine Kopie eines Dokuments und gibt den Inhalt als Stream-Objekt zurück.                                                                                                                | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja |
| Doc Methode<br>getDataObjectContents         | Ermöglicht den Zugriff auf den Inhalt des Dateianhangs, der einem Datenobjekt zugeordnet ist.                                                                                                     | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja |
| annotAttachment Eigenschaft<br>contentStream | Eine ReadStream-Eigenschaft, die den Dateianhang-Stream repräsentiert. Diese Eigenschaft ermöglicht den Zugriff auf den Inhalt des Dateianhangs, der einem annotAttachment-Objekt zugeordnet ist. | normal: Ja<br>dynamisch: Ja<br>eingebettet: Ja<br>privilegiert: Ja |

# Rechteausweitung

| API-Aufruf                  | Beschreibung                         | Reader Tests                                                               |
|-----------------------------|--------------------------------------|----------------------------------------------------------------------------|
| app Methode trustedFunction | Markiert eine Funktion als vertraut. | normal: Nein<br>dynamisch: Nein<br>eingebettet: Nein<br>privilegiert: Nein |